

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SÃO PAULO
CÂMPUS BIRIGUI

HENRIQUE JOSÉ DE SOUZA

SISTEMA ESPECIALISTA EDUCACIONAL PARA
VALIDAÇÃO DE COMPATIBILIDADE DE HARDWARE
BASEADO EM PROBLEMA DE SATISFAÇÃO DE
RESTRIÇÕES (CSP).

BIRIGUI

2026

HENRIQUE JOSÉ DE SOUZA

Sistema Especialista Educacional para Validação de
Compatibilidade de Hardware baseado em Problema de
Satisfação de Restrições (CSP).

Trabalho de Conclusão de Curso
apresentado ao Instituto Federal de
Educação, Ciência e Tecnologia de
São Paulo, como requisito parcial para
conclusão do curso de Engenharia da
Computação.

Área de Concentração: Área de
Concentração do Trabalho

Orientador: Profa. Dra. Helen de Freitas
Santos

BIRIGUI

2026

Dedicatória (Opcional). Não digite a palavra Dedicatória. Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém (não usar ponto final)

AGRADECIMENTOS

Agradecimento (opcional). Folha que contém manifestação de reconhecimento a pessoas e/ou instituições que realmente contribuíram com o autor, devendo ser expressos de maneira simples.

Epígrafe (Opcional). Pensamentos retirados de um livro, uma música, um poema, normalmente relacionado ao tema do trabalho, seguida de indicação de autoria. As epígrafes podem ser colocadas também nas folhas de abertura de cada capítulo.

“Any fool can write code that a computer can understand. Good programmers write code that humans can understand”.

Martin Fowler

RESUMO

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, fornecendo uma visão rápida e clara do conteúdo do estudo. O texto deverá conter entre 150 a 250 palavras e ser antecedido pela referência do estudo. Também, não deve conter citações e deverá ressaltar o objetivo, o método, os resultados e as conclusões. O resumo deve ser redigido em parágrafo único, seguido das palavras representativas do conteúdo do estudo, isto é, palavras-chave, em número de três a cinco, separadas entre si por ponto e finalizadas também por ponto. Usar o verbo na terceira pessoa do singular, com linguagem impessoal (pronomes SE), bem como fazer uso, preferencialmente, da voz ativa.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3. Palavra-chave n.

ABSTRACT OU RÉSUMÉ OU RESUMEN

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Keywords: Keyword 1. Keyword 2. Keyword 3. Keyword n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Mots-clés : Mot 1. Mot 2. Mot 3. Mot n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Palabras clave: Palabra 1. Palabra 2. Palabra 3. Palabra n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Grafo de Restrições	26
Figura 2 – Execução do AC-3	30
Figura 3 – Diagrama da arquitetura em três camadas do sistema	36
Figura 4 – Diagrama de Casos de Uso	70
Figura 5 – Modelo de Domínio	73
Figura 6 – Diagrama de Classes de Análise	74
Figura 7 – Tela inicial do sistema	76
Figura 8 – Tela de seleção de componentes com sinalização de incompatibilidade	77
Figura 9 – Tela de resumo da configuração finalizada	78
Figura 10 – Tela inicial em dispositivo móvel	79
Figura 11 – Tela de seleção de componentes em dispositivo móvel	80
Figura 12 – Tela de resumo da configuração em dispositivo móvel	81
Figura 13 – Diagrama Entidade-Relacionamento da base de conhecimento	85
Figura 14 – Diagrama de Sequência	87
Figura 15 – Arquitetura em Três Camadas do Sistema	88
Figura 16 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink	104

LISTA DE QUADROS

Quadro 1 – RF-01 – Listar componentes por categoria	49
Quadro 2 – RF-02 – Navegar pela interface <i>wizard</i>	50
Quadro 3 – RF-03 – Exibir componentes incompatíveis com sinalização visual . .	50
Quadro 4 – RF-04 – Exibir características técnicas dos componentes	51
Quadro 5 – RF-05 – Executar propagação de restrições via algoritmo AC-3	51
Quadro 6 – RF-06 – Validar compatibilidade de soquete CPU–Placa-mãe	52
Quadro 7 – RF-07 – Validar padrão de memória RAM–Placa-mãe	53
Quadro 8 – RF-08 – Validar capacidade energética da fonte (TDP)	54
Quadro 9 – RF-09 – Propagar restrições incrementalmente a cada seleção	55
Quadro 10 – RF-10 – Exibir justificativa técnica para cada incompatibilidade . . .	55
Quadro 11 – RF-11 – Exibir alerta de incompatibilidade de soquete	56
Quadro 12 – RF-12 – Exibir alerta de incompatibilidade de padrão de memória . .	56
Quadro 13 – RF-13 – Exibir alerta de fonte de alimentação subdimensionada	57
Quadro 14 – RF-14 – Reinicializar a configuração	57
Quadro 15 – RF-15 – Substituir componente selecionado em etapa anterior	58
Quadro 16 – RF-16 – Exibir resumo da configuração finalizada	58
Quadro 17 – RF-17 – Armazenar e consultar a base de conhecimento do CSP	59
Quadro 18 – RF-18 – Popular base de conhecimento via <i>Database Seeding</i>	60
Quadro 19 – RF-19 – Armazenar e consultar o conjunto de restrições (<i>C</i>)	61
Quadro 20 – RNF-01 – Tempo de resposta da validação	62
Quadro 21 – RNF-02 – Ausência de avaliação por força bruta	62
Quadro 22 – RNF-03 – Linguagem natural nas explicações de incompatibilidade . .	63
Quadro 23 – RNF-04 – Interface responsiva	63
Quadro 24 – RNF-05 – Acesso sem cadastro ou autenticação	64
Quadro 25 – RNF-06 – Separação estrita entre as três camadas arquiteturais	64
Quadro 26 – RNF-07 – Motor de inferência expansível para novas restrições	65
Quadro 27 – RNF-08 – Base de conhecimento genérica e reutilizável	65
Quadro 28 – RNF-09 – Restrições derivadas de documentação oficial dos fabricantes	66
Quadro 29 – RNF-10 – Determinismo do motor de inferência	66
Quadro 30 – RNF-11 – API <i>stateless</i> (sem estado de sessão no servidor)	67
Quadro 31 – RNF-12 – Ambiente de banco de dados reproduzível via Docker	67
Quadro 32 – RNF-13 – Rastreabilidade de cada bloqueio até a restrição violada . .	68
Quadro 33 – RNF-14 – Ausência de bloqueios silenciosos	68
Quadro 34 – Versões das principais tecnologias utilizadas	90
Quadro 35 – Registros criados pela carga inicial da base de conhecimento	92
Quadro 36 – Recursos expostos pela API REST	99

LISTA DE TABELAS

Tabela 1 – Modelo para Especificação de Casos de Uso	71
Tabela 2 – Especificação do Caso de Uso “Selecionar Componente”	72
Tabela 3 – Convenção para Nome dos Objetos no Banco de Dados	83
Tabela 4 – Tabelas Identificadas neste Trabalho	83
Tabela 5 – Tipos Enumerados do Modelo de Dados	83
Tabela 6 – Marca	84
Tabela 7 – Categoria	84
Tabela 8 – Caracteristica	84
Tabela 9 – Componente	84
Tabela 10 – ComponenteCaracteristica	84
Tabela 11 – Restricao	84

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1	INTRODUÇÃO	16
1.1	Justificativa	17
1.2	Objetivos	18
1.2.1	Objetivo Geral	18
1.2.2	Objetivos Específicos	18
1.3	Problema (Hipótese ou Questão de Pesquisa)	19
1.4	Organização Deste Trabalho	19
2	REVISÃO DA LITERATURA	21
2.1	Compatibilidade de Hardware como Domínio do Problema	21
2.1.1	Compatibilidade de Soquete CPU-Placa-mãe	21
2.1.2	Padrão de Memória RAM	21
2.1.3	Capacidade Energética e TDP	22
2.1.4	Formalização como Problema de Restrições	22
2.2	Problemas de Satisfação de Restrições (CSP)	22
2.2.1	Tipos de Restrições	23
2.2.2	Aplicação ao Domínio de Hardware	23
2.2.3	Representação em Grafo	24
2.3	Teoria dos Grafos e Grafo de Restrições	24
2.3.1	Definições Fundamentais	24
2.3.2	Representações Computacionais	24
2.3.3	Grafo de Restrições	25
2.4	Complexidade Computacional	26
2.4.1	Implicações para o Domínio de Hardware	27
2.5	Algoritmos de Busca e Backtracking	27
2.5.1	Busca em Espaço de Atribuições	27
2.5.2	Backtracking	28
2.5.3	Forward Checking	29
2.6	Propagação de Restrições e Algoritmo AC-3	29
2.6.1	Aplicação ao Domínio de Hardware	29
2.7	Sistema Especialista	31
2.7.1	Explanation Facilities	31
2.7.2	Inteligência Artificial Explicável (XAI)	32
2.7.3	Mapeamento ao Sistema Proposto	32
2.8	Dimensão Educativa em Sistemas Especialistas	33

2.8.1	Mecanismo de Filtragem versus Sistema com Explicação	33
2.8.2	Sistemas Tutores Inteligentes	33
2.8.3	Sistema Especialista Educacional	34
2.9	Arquitetura de Software Orientada a Serviços	34
2.9.1	Arquitetura em Camadas	34
2.9.2	REST como Estilo Arquitetural	35
2.9.3	Mapeamento ao Sistema Proposto	35
2.10	Trabalhos Relacionados	37
2.10.1	Plataformas Comerciais de Compatibilidade de Hardware	37
2.10.2	Trabalhos Acadêmicos Relacionados	37
2.10.3	Posicionamento do Trabalho Proposto	38
3	MATERIAIS E MÉTODOS	39
3.0.1	Bibliográfica	39
3.0.2	Aplicada	40
3.1	Materiais	40
3.1.1	Equipamentos	40
3.1.2	Softwares	40
3.2	Ferramentas Utilizadas	41
3.2.1	Visual Studio Code	41
3.2.2	Node.js e NestJS	41
3.2.3	PostgreSQL e Docker	41
3.2.4	DBeaver	42
3.2.5	Next.js	42
3.2.6	Figma	42
3.2.7	Astah Community	43
3.2.8	Teste?	43
3.3	Métodos de Desenvolvimento	43
3.3.1	Modelagem do Problema como CSP	43
3.3.2	Implementação do Algoritmo AC-3	43
3.3.3	Implementação das Justificativas Educativas	44
3.3.4	Processo de Desenvolvimento de Software	44
3.3.5	Estratégia de Testes	45
4	DESCRIÇÃO GERAL DO PRODUTO	46
4.1	Funcionalidades Principais	46
4.2	Diferencial do Produto	47
5	ELICITAÇÃO DE REQUISITOS E ANÁLISE	48
5.1	Requisitos do Usuário	48

5.2	Requisitos do Sistema	48
5.2.1	Requisitos Funcionais	49
5.2.2	Requisitos Não Funcionais	61
5.2.3	Restrições, Suposições e Dependências	68
5.2.4	Requisitos Adiados	69
5.3	Casos de Uso	69
5.3.1	Diagrama de Casos de Uso	69
5.3.2	Especificação dos Casos de Uso	70
5.4	Modelo de Domínio	73
5.5	Diagrama de Classes de Análise	73
5.6	Diagrama de Atividades	74
6	PROJETO DE SOFTWARE	75
6.1	Projeto de Interface	75
6.2	Projeto de Dados	82
6.2.1	Mapeamento Objeto-Relacional	82
6.2.2	Estrutura das Tabelas no Banco de Dados	82
6.2.3	Diagrama de Pacotes	86
6.2.4	Diagrama de Classes de Projeto	86
6.3	Projeto Procedimental	86
6.3.1	Diagrama de Sequência	86
6.4	Projeto Arquitetural	87
7	IMPLEMENTAÇÃO	89
7.1	Ambiente e Ferramentas de Desenvolvimento	89
7.2	Organização do Repositório	90
7.3	Camada de Dados	91
7.4	Camada de Lógica de Negócio	93
7.4.1	Isolamento do Motor de Inferência	93
7.4.2	Carregamento e Montagem do Grafo	93
7.4.3	O Algoritmo AC-3	95
7.4.4	Avaliação Genérica de Restrições	96
7.4.5	Geração das Justificativas Educativas	97
7.5	Camada de Apresentação	98
7.6	A API REST	99
7.7	Execução do Ambiente	99
7.8	Limitações Conhecidas da Implementação	100
8	TESTE	102
9	IMPLANTAÇÃO	105

10	RESULTADOS E DISCUSSÃO	106
11	CONCLUSÕES PARCIAIS	107
12	CRONOGRAMA	109
	Referências	110

1 INTRODUÇÃO

A montagem de um computador *desktop* exige o cruzamento simultâneo de especificações técnicas entre componentes interdependentes. Diferentemente da aquisição de periféricos genéricos, a configuração de um sistema computacional envolve restrições físicas, elétricas e lógicas que, quando violadas, resultam em incompatibilidade absoluta, os componentes adquiridos de forma incorreta simplesmente não funcionam juntos, independentemente de ajustes de *software* ou configuração. A compatibilidade de soquete entre processador e placa-mãe, o padrão de memória suportado pela plataforma e o dimensionamento energético da fonte de alimentação são exemplos de restrições binárias e determinísticas que não admitem adaptações. Esse cenário impõe ao usuário um desafio técnico considerável, uma vez que a violação de qualquer dessas restrições representa prejuízo financeiro imediato e frustrante.

Diante dessa realidade, plataformas como o PC Part Picker e o Meupc.net se estabeleceram como referências globais e nacionais para a verificação de compatibilidade entre componentes. Ambas detectam automaticamente conflitos entre os itens selecionados e alertam o usuário quando uma incompatibilidade é identificada (PCPartPicker LLC, 2024; Meupc.net, 2024). No entanto, quando o conflito é detectado, a resposta do sistema se limita a sinalizar que os componentes não são compatíveis entre si, sem indicar a regra física que foi violada. O usuário aprende o que não pode fazer, mas não aprende o motivo, e portanto, não está em condições de generalizar esse conhecimento para outras situações, como a escolha de um processador diferente ou a adição de um módulo de memória RAM.

Essa limitação não é acidental, ela decorre da arquitetura dessas plataformas que operam como mecanismos de filtragem e não como sistemas especialistas com capacidade de explicação. Segundo Clancey (1983), a capacidade de justificar o próprio raciocínio é o que distingue fundamentalmente um sistema especialista de um mecanismo de filtragem simples. Um filtro responde apenas à pergunta "*quais opções são válidas?*", enquanto um sistema especialista dotado de *explanation facility* responde também à pergunta "*por que as demais são inválidas?*", articulando a cadeia de regras e restrições que levou aquela conclusão. Nesse sentido, cada bloqueio silencioso de um mecanismo de filtragem é uma oportunidade de aprendizagem perdida ao usuário.

Do ponto de vista da ciência da computação, o problema de configuração de *hardware* recai formalmente sobre a classe dos Problemas de Satisfação de Restrições (CSP). Nessa formalização, cada categoria de componente corresponde a uma variável, o conjunto de modelos comercialmente disponíveis constitui o domínio dessa variável e as regras físicas de compatibilidade entre pares de componentes definem as restrições do

problema (Russell; Norvig, 2010). O CSP é classificado como NP-completo (Sipser, 2012), o que torna a avaliação exaustiva de todas as combinações inviável para catálogos de tamanho realista. Técnicas de propagação de restrições, como o algoritmo AC-3 proposto por Mackworth (1977), contornam essa explosão combinatória reduzindo o espaço de busca de forma incremental a cada interação do usuário, tornando a validação viável em milissegundos.

Neste contexto, este Trabalho de Conclusão de Curso tem como objetivo central abordar a lacuna identificada, desenvolvendo um sistema especialista educacional para validação de compatibilidade de *hardware* fundamentado em CSP. O sistema proposto é uma aplicação *web full-stack* capaz de validar a compatibilidade entre componentes de computador *desktop* e, para cada incompatibilidade detectada, comunicar ao usuário a regra técnica violada em linguagem natural. Ao transformar cada bloqueio em uma oportunidade de aprendizagem, o sistema busca não apenas restringir escolhas incorretas, mas educar o usuário sobre as restrições físicas que regem a montagem de computadores.

1.1 Justificativa

A montagem de um computador *desktop* exige do usuário o domínio de um conjunto de especificações técnicas que, na prática, está além do conhecimento da maioria dos consumidores. As restrições que regem a compatibilidade entre componentes são objetivas e documentadas pelos próprios fabricantes Advanced Micro Devices (2022), Intel Corporation (2023) e JEDEC Solid State Technology Association (2020), mas sua compreensão pressupõe familiaridade com conceitos como soquetes de processador, padrões de memória e dimensionamento energético. Para o usuário leigo, essa barreira técnica se manifesta de forma concreta, erros de seleção, como a combinação de um processador AM5 com uma placa-mãe AM4 ou de módulo DDR4 com uma plataforma que suporta exclusivamente DDR5, resultam em incompatibilidade absoluta e em prejuízo financeiro.

As plataformas comerciais disponíveis atualmente buscam mitigar esse problema, mas com uma abordagem limitada. O PC Part Picker e o Meupc.net realizam verificações de compatibilidade e alertam o usuário quando um conflito é detectado (PCPartPicker LLC, 2024; Meupc.net, 2024). No entanto, como demonstrado por Clancey (1983), a limitação pedagógica desse modelo é nítida, um sistema que apenas filtra não transfere conhecimento, apenas restringe escolhas. Um usuário que utiliza essas plataformas repetidamente aprende quais componentes são compatíveis entre si, mas não aprende "*por que*" são compatíveis, o que o impede de generalizar o conhecimento adquirido para novas situações de compra.

Essa lacuna motivou o questionamento central deste trabalho: seria possível desenvolver um sistema que realizasse a validação de compatibilidade de *hardware* com um motor de inferência formal, baseado no formalismo de CSP, e que comunicasse ao usuário cada

incompatibilidade detectada por meio de uma justificativa técnica em linguagem natural? A resposta a essa pergunta orienta todo o desenvolvimento deste trabalho, que propõe não apenas um verificador de compatibilidade, mas um sistema especialista educacional cuja finalidade é transformar cada bloqueio em uma oportunidade concreta de aprendizagem sobre os princípios que regem a montagem de computadores.

Além da relevância prática imediata para o usuário leigo, o trabalho justifica-se do ponto de vista acadêmico por articular, em uma única solução, dois campos consolidados da ciência da computação: o formalismo dos Problemas de Satisfação de Restrições, com o algoritmo AC-3 como motor de propagação, e a teoria dos sistemas especialistas, com sua capacidade de explicação. O domínio de compatibilidade de *hardware* é adotado como estudo de caso por ser concreto, verificável a partir da documentação oficial dos fabricantes e de imediata utilidade, servindo de veículo para materializar uma arquitetura genérica que, por decisão de projeto, não fica restrita a esse domínio.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver um sistema especialista educacional para validação de compatibilidade de *hardware* de computadores *desktop*, fundamentado no formalismo de Problemas de Satisfação de Restrições (CSP) e no algoritmo de propagação AC-3, capaz não apenas de detectar incompatibilidades entre componentes, mas de comunicar ao usuário leigo, em linguagem natural e acessível, a razão técnica de cada incompatibilidade, transformando cada bloqueio em uma oportunidade de aprendizagem.

1.2.2 Objetivos Específicos

- modelar o problema de compatibilidade de *hardware* como um Problema de Satisfação de Restrições, adotando uma representação de conhecimento genérica e dirigida a dados que dissocie o motor de inferência do domínio específico de aplicação;
- implementar o algoritmo AC-3 como motor de inferência para a propagação incremental de restrições, operando de forma agnóstica ao domínio sobre um grafo de restrições construído dinamicamente a partir da base de conhecimento;
- dotar o sistema de uma *explanation facility* que gere, para cada restrição violada, uma justificativa técnica em linguagem natural, alinhada aos princípios da Inteligência Artificial Explicável (XAI);

- aplicar e validar o motor de inferência no domínio de compatibilidade de *hardware desktop* como estudo de caso, modelando suas restrições de soquete, padrão de memória e capacidade energética como instâncias do CSP;
- desenvolver uma interface *web* no formato *wizard* que evidencie o caráter educativo da solução, mantendo os componentes incompatíveis visíveis e acompanhados da respectiva justificativa, em vez de simplesmente ocultá-los.

1.3 Problema (Hipótese ou Questão de Pesquisa)

Este trabalho parte da seguinte questão de pesquisa: é possível construir um sistema especialista, fundamentado no formalismo de CSP e no algoritmo AC-3, que não apenas valide a compatibilidade entre componentes de *hardware* de forma correta e eficiente, mas que também ensine o usuário leigo, comunicando em linguagem natural a razão técnica de cada incompatibilidade detectada?

Da questão central derivam-se dois questionamentos complementares. O primeiro, de natureza técnica, indaga se um motor de inferência baseado em propagação de restrições, com o conhecimento do domínio representado como dados e não como regras codificadas, é capaz de validar configurações de *hardware* sem recorrer à avaliação exaustiva por força bruta, inviável dada a natureza NP-completa do CSP. O segundo, de natureza pedagógica, indaga se a substituição do bloqueio silencioso, característico dos mecanismos de filtragem, por uma justificativa técnica explícita é suficiente para transformar a interação de validação em uma experiência educativa, sem exigir do usuário qualquer cadastro, histórico ou conhecimento técnico prévio.

1.4 Organização Deste Trabalho

Além desta introdução, este trabalho está organizado em mais onze capítulos. O Capítulo 2 apresenta a revisão da literatura, abordando o domínio de compatibilidade de *hardware*, o formalismo dos Problemas de Satisfação de Restrições, a teoria dos grafos, a complexidade computacional, os algoritmos de busca e propagação de restrições, os sistemas especialistas e sua dimensão educativa, a arquitetura de *software* orientada a serviços e os trabalhos relacionados. O Capítulo 3 descreve os materiais e métodos empregados, detalhando as metodologias de pesquisa, as ferramentas utilizadas e os métodos de desenvolvimento adotados.

O Capítulo 4 apresenta a descrição geral do produto desenvolvido. O Capítulo 5 detalha a elicitação e a análise dos requisitos, incluindo os requisitos funcionais e não funcionais, os casos de uso e os modelos de análise. O Capítulo 6 apresenta o projeto de *software*, contemplando os projetos de interface, de dados, procedimental e arquitetural.

Os capítulos seguintes tratam da concretização e da avaliação do sistema: o Capítulo 7 descreve a implementação; o Capítulo 8, os testes realizados; e o Capítulo 9, a implantação. O Capítulo 10 apresenta os resultados e a discussão, e o Capítulo 11 expõe as conclusões parciais obtidas até a etapa de Validação de Projeto. Por fim, o Capítulo 12 apresenta o cronograma das atividades previstas até a Avaliação Final.

2 REVISÃO DA LITERATURA

2.1 Compatibilidade de Hardware como Domínio do Problema

A montagem de um computador *desktop* exige o cruzamento de especificações técnicas entre componentes interdependentes. Diferentemente da aquisição de periféricos genéricos, a configuração de um sistema computacional envolve restrições físicas, elétricas e lógicas que, quando violadas, resultam em incompatibilidade absoluta: componentes adquiridos de forma incorreta não funcionam juntos, independentemente de ajustes de *software* ou configuração. Esse cenário impõe ao usuário leigo um desafio técnico significativo, uma vez que plataformas comerciais tradicionais não oferecem mecanismos de validação preventiva capazes de explicar o motivo da incompatibilidade.

O presente trabalho concentra-se em cinco categorias de componentes que constituem o núcleo de qualquer configuração *desktop*: a Unidade Central de Processamento (CPU), a placa-mãe, os módulos de memória RAM, a Unidade de Processamento Gráfico (GPU) e a fonte de alimentação. Cada uma dessas categorias possui especificações técnicas que determinam sua compatibilidade com as demais, formando uma rede de dependências que não pode ser avaliada de forma isolada.

2.1.1 Compatibilidade de Soquete CPU-Placa-mãe

A restrição mais fundamental na montagem de um computador é a compatibilidade de soquete entre o processador e a placa-mãe. Cada geração de processadores é projetada para um soquete físico específico, definido pelo número, disposição e função de seus contatos elétricos. Essa restrição é binária e absoluta: um processador projetado para o soquete AM5 da AMD é fisicamente incompatível com uma placa-mãe equipada com soquete AM4, e vice-versa, sem qualquer possibilidade de adaptação ou ajuste (Advanced Micro Devices, 2022). De forma análoga, processadores Intel para o soquete LGA1851 não são compatíveis com plataformas LGA1700 (Intel Corporation, 2023). A documentação técnica oficial dos fabricantes define formalmente quais combinações de processadores e *chipsets* são suportadas, constituindo a principal fonte de restrições binárias do domínio.

2.1.2 Padrão de Memória RAM

Os módulos de memória RAM seguem padrões técnicos definidos pela JEDEC (*Joint Electron Device Engineering Council*), organização responsável pela padronização internacional de semicondutores. Os padrões DDR4 e DDR5, por exemplo, diferem em números de pinos, tensão de operação, frequência base e formato físico do encaixe (JEDEC

Solid State Technology Association, 2020). Essa diferença torna os dois padrões eletricamente e fisicamente incompatíveis: uma placa-mãe projetada para DDR5 não aceita módulos DDR4, e o próprio conector possui geometria distinta que impede a inserção física incorreta. A seleção de memória compatível depende, portanto, da placa-mãe selecionada, que por sua vez depende do processador, criando uma cadeia de dependências transitivas no grafo de restrições.

2.1.3 Capacidade Energética e TDP

Cada componente de *hardware* possui um indicador de consumo máximo de energia denominado TDP (*Thermal Design Power*), expresso em watts. A fonte de alimentação deve ser dimensionada para suprir a soma dos TDPs de todos os componentes, com uma margem de segurança recomendada de aproximadamente 20% a 30% acima do consumo estimado (Intel Corporation, 2022). Uma fonte subdimensionada provoca instabilidade do sistema, reinicializações inesperadas e, em casos extremos, danos permanentes aos componentes. Essa restrição difere das anteriores por ser de natureza quantitativa: não se trata de uma incompatibilidade binária, mas de uma relação de desigualdade entre a capacidade da fonte e a demanda agregada dos componentes.

2.1.4 Formalização como Problema de Restrições

As três categorias de restrições descritas, soquete, padrão de memória e capacidade energética, possuem uma estrutura comum: cada um envolve um par de componentes e define condições que os valores atribuídos a esse par devem satisfazer. Essa estrutura é precisamente o que a teoria dos Problemas de Satisfação de Restrições (CSP) formaliza: cada categoria de componente pode ser modelada como uma variável, o conjunto de modelos comercialmente disponíveis em cada categoria constitui o domínio dessa variáveis. Essa correspondência torna o CSP não apenas uma metáfora conveniente, mas a estrutura formal mais adequada para modelar o problema de validação de configurações de *hardware*, conforme detalhado na seção seguinte.

2.2 Problemas de Satisfação de Restrições (CSP)

Um Problema de Satisfação de Restrições (CSP) é formalmente caracterizado por uma tripla $\langle X, D, C \rangle$, na qual $X = \{X_1, \dots, X_n\}$ denota um conjunto finito de variáveis, $D = \{D_1, \dots, D_n\}$ representa os respectivos domínios, de modo que cada D_i delimita os valores admissíveis para X_i , e C reúne as restrições que impõem condições sobre as combinações válidas de valores (Russell; Norvig, 2010). Cada restrição C_j é expressa como um par $\langle \text{scope}, \text{rel} \rangle$, em que *scope* é uma tupla de variáveis participantes e *rel* define os valores que essas variáveis podem assumir conjuntamente (Russell; Norvig, 2010). Diz-se

que o problema está resolvido quando se obtém uma atribuição completa e consistente: uma função que associa a cada variável um valor de seu domínio sem infringir nenhuma restrição de C .

2.2.1 Tipos de Restrições

As restrições em um CSP podem ser classificadas quanto ao número de variáveis que envolvem. Restrições unárias afetam uma única variável, limitando seu domínio independentemente das demais, por exemplo: a restrição de que a fonte de alimentação deve ter potência mínima de 500W. Restrições binárias envolvem duas variáveis e são as mais comuns em problemas de compatibilidade de *hardware*, por exemplo: a restrição de que o soquete da CPU deve corresponder ao soquete suportado pela placa-mãe. Restrições de ordem superior envolvem três ou mais variáveis simultaneamente (Dechter, 2003). O sistema proposto neste trabalho opera predominantemente com restrições binárias, o que viabiliza a representação estrutural em grafo de restrições.

2.2.2 Aplicação ao Domínio de Hardware

A aplicação dessa formalização ao domínio de compatibilidade de *hardware* é direta. Considerando os cinco componentes centrais do sistema (CPU, placa-mãe, RAM, GPU e fonte), a tripla (X, D, C) pode ser definida da seguinte forma:

- $X = \{\text{CPU, Placa-mãe, RAM, GPU, Fonte}\};$
- $D = \{D_{\text{CPU}} = \text{modelos de processadores disponíveis,}$
 $D_{\text{Placa}} = \text{modelos de placas-mãe disponíveis,}$
 $D_{\text{RAM}} = \text{modelos de memória RAM disponíveis,}$
 $D_{\text{GPU}} = \text{modelos de placas de vídeo disponíveis,}$
 $D_{\text{Fonte}} = \text{modelos de fontes disponíveis}\}$
- $C = \{\text{soquete}(\text{CPU, Placa-mãe}),$
 $\text{padrão_memória}(\text{Placa-mãe, RAM}),$
 $\text{tdp_cpu}(\text{CPU, Fonte}),$
 $\text{tdp_gpu}(\text{GPU, Fonte})\}$

Para ilustrar concretamente, considere a atribuição parcial $\{\text{CPU} = \text{Ryzen 5 7600, Placa-mãe} = \text{B650, RAM} = \text{DDR5-5600}\}$. Essa atribuição é consistente: o Ryzen 5 7600 utiliza soquete AM5, a placa B650 suporta AM5, e o padrão DDR5 é compatível com a plataforma AM5. Contudo, a atribuição $\{\text{CPU} = \text{Ryzen 5 7600, Placa-mãe} = \text{B650, RAM} = \text{DDR4-3200}\}$ viola a restrição padrão_memória , pois placas-mãe com chipset

B650 suportam exclusivamente DDR5 (Advanced Micro Devices, 2022; JEDEC Solid State Technology Association, 2020). A solução do CSP consiste em encontrar uma atribuição completa, cobrindo todas as cinco variáveis, que satisfaça simultaneamente todas as restrições de C .

2.2.3 Representação em Grafo

A principal distinção entre o CSP e as abordagens clássicas de busca em espaço de estados reside na representação fatorada que aquele adota: em vez de tratar o estado como uma entidade atômica e indivisível, o CSP expõe a estrutura interna do problema por meio de variáveis e restrições explícitas (Russell; Norvig, 2010; Kumar, 1992). Restrições binárias entre pares de variáveis admitem representação natural como grafo, no qual os vértices correspondem às variáveis e as arestas codificam as restrições. Essa estrutura de grafo de restrições é o objeto central sobre o qual os algoritmos de busca e propagação operam.

2.3 Teoria dos Grafos e Grafo de Restrições

2.3.1 Definições Fundamentais

Um grafo é formalmente definido como um par ordenado $G = (V, E)$, onde V é um conjunto finito não vazio de vértices e $E \subseteq V \times V$ é um conjunto de arestas (Diestel, 2017). Um grafo é dito simples quando não contém laços (arestas de um vértice para si mesmo) nem arestas paralelas (múltiplas arestas entre o mesmo par de vértices). Em um grafo não direcionado, as arestas não possuem orientação: a aresta $\{u, v\}$ é idêntica a $\{v, u\}$. Dois vértices u e v são ditos adjacentes se existe uma aresta $\{u, v\} \in E$ conectando-os (Diestel, 2017).

2.3.2 Representações Computacionais

Do ponto de vista computacional, um grafo pode ser representado de duas formas principais (Cormen *et al.*, 2001). A matriz de adjacência é uma matriz A de dimensão $|V| \times |V|$, na qual $A[i][j] = 1$ se existe aresta entre os vértices i e j , e 0 caso contrário. Essa representação permite verificar a existência de uma aresta em tempo constante $O(1)$, mas exige espaço $O(|V|^2)$, independentemente do número de arestas. A lista de adjacência associa a cada vértice uma lista de seus vizinhos, exigindo espaço $O(|V| + |E|)$. Para grafos esparsos, onde o número de arestas é significativamente menor que V^2 , a lista de adjacência é preferível em termos de eficiência de memória (Cormen *et al.*, 2001).

O grafo de restrições do sistema proposto é esparso por natureza: com cinco vértices (componentes), o número máximo de arestas seria 10, mas o sistema possui apenas quatro

restrições binárias efetivas, soquete entre CPU e Placa-mãe, padrão de memória entre Placa-mãe e RAM, TDP da CPU e Fonte, e TDP da GPU e Fonte. Pares como GPU e RAM ou CPU e GPU, não possuem restrições física direta e portanto não são conectados por arestas. A lista de adjacência é, portanto, a representação mais adequada para esse grafo.

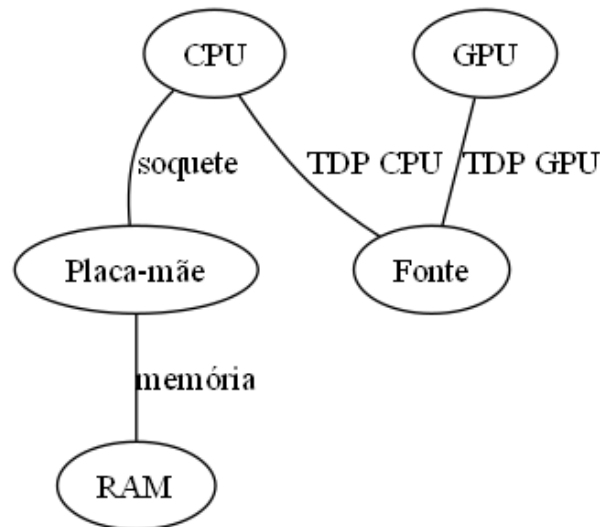
2.3.3 Grafo de Restrições

Dado um CSP com variáveis X e restrições C , o grafo de restrições $G_C = (X, E)$ é construído de modo que existe uma aresta $\{X_i, X_j\} \in E$ se e somente se existe alguma restrição C envolvendo simultaneamente as variáveis X_i e X_j (Russell; Norvig, 2010). Essa representação explicita a estrutura topológica do problema, vértices isolados correspondem a variáveis sem restrições com outras, e componentes fortemente conectados indicam grupos de variáveis com alta interdependência.

Aplicando essa definição ao sistema proposto, o grafo de restrições possui a seguinte estrutura:

- **Vértices:** $V = \{\text{CPU, Placa-mãe, RAM, GPU, Fonte}\}$
- **Arestas:** E compreende as seguintes relações:
 - $\{\text{CPU, Placa-mãe}\}$, restrição de soquete;
 - $\{\text{Placa-mãe, RAM}\}$, restrição de padrão de memória;
 - $\{\text{CPU, Fonte}\}$, restrição de TDP do processador;
 - $\{\text{GPU, Fonte}\}$, restrição de TDP da placa de vídeo.

Figura 1 – Grafo de Restrições do Sistema Proposto



Fonte: Elaborada pelo autor

Essa estrutura revela que a Fonte é um vértice de alta centralidade no grafo, pois está conectada tanto à CPU quanto à GPU, de modo que qualquer alteração no domínio de um desses componentes pode propagar restrições para o domínio da Fonte. A Placa-mãe, por sua vez, é o vértice que medeia a compatibilidade entre CPU e RAM, funcionando como ponto de articulação da rede de restrições. Esse grafo é o objeto sobre o qual os algoritmos de busca e propagação operam, cuja complexidade computacional é analisada na seção seguinte.

2.4 Complexidade Computacional

A teoria da complexidade computacional ocupa-se da classificação de problemas de decisão segundo os recursos algorítmicos, principalmente tempo de execução e espaço de memória, exigidos para sua resolução (Cormen *et al.*, 2001). Dentro dessa estrutura, a classe P reúne os problemas para os quais existem algoritmos determinísticos de tempo polinomial, considerados eficientes e tratáveis mesmo para entradas de grande escala. A classe NP, por sua vez, abrange os problemas cuas soluções propostas podem ser verificados em tempo polinomial por uma máquina determinística, ainda que não se conheça, até o presente, algoritmo eficiente para encontrá-las (Sipser, 2012). A questão sobre a relação entre essas duas classes, se $P = NP$, permanece em aberto e constitui o problema não resolvido de maior relevância para a ciência da computação teórica, razão pela qual o *Clay Mathematics Institute* o incluiu entre os sete Problemas do Milênio (Sipser, 2012).

O interior da classe NP não é uniforme. A literatura demonstra a existência de subclasses com diferentes graus de dificuldade, incluindo problemas resolúveis em tempo subexponencial por meio de não-determinismo limitado (Papadimitriou; Yannakakis,

1996). O CSP, contudo, não se enquadra nessas classes intermediárias, situando-se no topo da hierarquia como um problema NP-completo (Sipser, 2012). Um problema é classificado como NP-completo quando pertence a NP e todo problema NP se reduz a ele em tempo polinomial, essa classificação pode ser estabelecida por redução polinomial a partir do Problema da Satisfabilidade Booleana (SAT), um dos problemas canônicos de NP-completude (Cormen *et al.*, 2001; Sipser, 2012). Consequentemente, se qualquer problema NP-completo admitisse solução em tempo polinomial, toda classe NP colapsaria em P (Sipser, 2012).

2.4.1 Implicações para o Domínio de Hardware

Essa natureza NP-completa impõe restrições severas à arquitetura de qualquer sistema de verificação de compatibilidade de *hardware* fundamentado em CSP. A primeira limitação é de ordem temporal, a avaliação exaustiva de todas as combinações interdependentes de componentes é inviável na prática em razão da explosão combinatória intrínseca à classe (Sipser, 2012; Cormen *et al.*, 2001). Para ilustrar concretamente, considere que cada categoria de componente possui em média 50 modelos disponíveis no catálogo do sistema. Com cinco categorias (CPU, Placa-mãe, RAM, GPU e Fonte), a avaliação por força bruta implicaria examinar $50^5 = 312.500.000$ atribuições candidatas antes de encontrar uma solução válida no pior caso. Esse número cresce exponencialmente à medida que o catálogo se expande, tornando a abordagem inviável em tempo hábil.

A segunda limitação é de ordem espacial, estratégias alternativas baseadas no mapeamento estático e exaustivo de regras lógicas condicionais, codificando explicitamente cada restrição de *hardware* por meio de estruturas if-else ou tabela de decisão, não contornam o problema, pois a representação de funções lógicas complexas por árvores de decisão pode exigir espaço de tamanho exponencial (Jukna *et al.*, 1999). Diante dessas limitações, a abordagem padrão para tornar CSPs tratáveis na prática é a busca sistemática com retrocesso combinada com técnicas de propagação de restrições, que reduzem o espaço efetivo de busca de forma incremental.

2.5 Algoritmos de Busca e Backtracking

2.5.1 Busca em Espaço de Atribuições

Dado o grafo de restrições e a complexidade NP-completa do CSP, é necessário um algoritmo de busca sistemática que explore o espaço de atribuições de forma inteligente, sem enumerar cegamente todas as combinações possíveis. O ponto de partida é o conceito de atribuição parcial, uma função que associa valores a um subconjunto das variáveis, podendo ser progressivamente estendida até uma atribuição completa (Russell; Norvig, 2010). A ideia central dos algoritmos de busca para CSP é construir a atribuição incrementalmente,

verificando consistência a cada passo e abandonando ramos da árvore de busca assim que uma inconsistência é detectada.

2.5.2 Backtracking

O algoritmo de *backtracking* é o método fundamental de busca para CSPs (Russell; Norvig, 2010; Kumar, 1992). Ele percorre recursivamente a árvore de atribuições, em cada nível da recursão, seleciona uma variável ainda não atribuída, tenta cada valor do seu domínio na ordem disponível, verifica se esse valor é consistente com as restrições envolvendo as variáveis já atribuídas e recua (*backtrack*) ao encontrar uma inconsistência. O pseudocódigo do algoritmo é apresentado a seguir:

Algoritmo 1 – Algoritmo de Busca por Backtracking para CSP

```

1: função BACKTRACKING-SEARCH(csp)
2:   retornar BACKTRACK( $\emptyset$ , csp)
3: fim função

4: função BACKTRACK(atribuicao, csp)
5:   se atribuicao está completa então
6:     retornar atribuicao
7:   fim se
8:   var  $\leftarrow$  SELECIONAR-VARIAVEL-NAO-ATRIBUÍDA(csp)
9:   para cada valor em ORDENAR-VALORES(var, atribuicao, csp) faça
10:    se valor é consistente com atribuicao dado csp então
11:      adicionar  $\{var = valor\}$  à atribuicao
12:      resultado  $\leftarrow$  BACKTRACK(atribuicao, csp)
13:      se resultado  $\neq$  falha então
14:        retornar resultado
15:      fim se
16:      remover  $\{var = valor\}$  da atribuicao
17:    fim se
18:  fim para
19:  retornar falha
20: fim função

```

Fonte: Adaptado de: Russell e Norvig (2010)

A complexidade de pior caso do *backtracking* puro é $O(d^n)$, na qual d é o tamanho máximo dos domínios e n é o número de variáveis, reiterando a inviabilidade para instâncias grandes sem otimizações complementares (Russell; Norvig, 2010).

2.5.3 Forward Checking

Uma primeira melhoria ao backtracking puro é o *forward checking* (Dechter, 2003). Ao atribuir um valor à variável X_i , o algoritmo imediatamente examina cada variável X_j , ainda não atribuída que compartilha uma restrição com X_i e remove de $Dom(X_j)$ todos os valores que se tornam inconsistentes com a atribuição de X_i . Se algum domínio X_j se torna vazio, o algoritmo detecta a falha antecipadamente e recua, sem precisar explorar o ramo até o fim.

Aplicando ao domínio de *hardware*, ao selecionar uma CPU com soquete AM5, o *forward checking* percorre os vizinhos de CPU no grafo de restrições e remove do domínio de Placa-mãe todos os modelos com soquete incompatível com AM5, antes mesmo de tentar atribuir um valor a Placa-mãe. Essa poda antecipada reduz significativamente o número de atribuições exploradas. Contudo, o *forward checking* verifica apenas os vizinhos diretos da variável recém-atribuída, sem propagar as consequências dessas remoções para variáveis mais distantes no grafo. A forma mais completa e eficiente de propagação, que garante consistência global na rede, é o Algoritmo AC-3 (Mackworth, 1977; Russell; Norvig, 2010).

2.6 Propagação de Restrições e Algoritmo AC-3

A principal técnica para contornar a explosão combinatória é a propagação de restrições, realizada por algoritmos de consistência. O mais consolidado na literatura é o AC-3 (*Arc Consistency Algorithm 3*), proposto por (Mackworth, 1977), cujo objetivo é garantir a consistência de arco em redes de restrições binárias (Kumar, 1992). Diz-se que um arco (X_i, X_j) é consistente quando, para todo valor $v \in Dom(X_i)$, existe ao menos um valor $w \in Dom(X_j)$ tal que a atribuição $X_i = v, X_j = w$ satisfaz a restrição entre as duas variáveis.

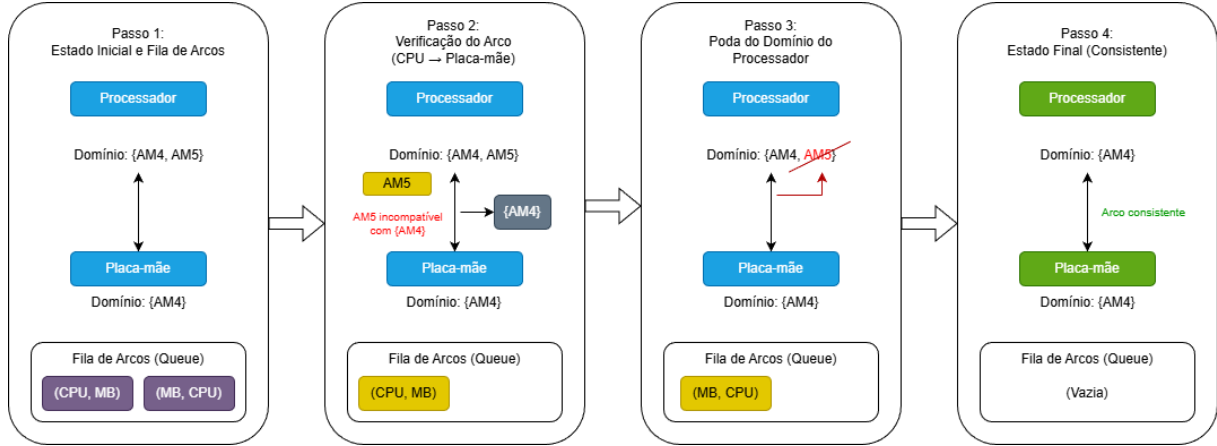
O algoritmo mantém uma fila de arcos a examinar e, para cada par de variáveis relacionadas, remove do domínio da variável os valores que não encontram suporte em nenhum valor do domínio da variável vizinha. Cada remoção dispara a reinserção dos arcos afetados na fila, propagando a mudança pela rede até o ponto de estabilidade, em que nenhuma redução adicional é possível (Russell; Norvig, 2010). Assumindo um CSP com n variáveis, domínios de tamanho máximo d e c restrições binárias, a complexidade de pior caso do AC-3 é $O(cd^3)$ (Russell; Norvig, 2010).

2.6.1 Aplicação ao Domínio de Hardware

Para ilustrar a execução do AC-3 no sistema proposto, considera-se o cenário em que o usuário seleciona um processador com soquete AM5. A Figura 2 apresenta os

quatro passos da execução do algoritmo sobre o grafo de restrições formado pelas variáveis Processador e Placa-mãe.

Figura 2 – Execução passo a passo do AC-3 no grafo de hardware



Fonte: Elaborada pelo autor

No **Passo 1**, o estado inicial é estabelecido: o domínio do Processador contém dois valores, AM4 e AM5, e o domínio da Placa-mãe contém apenas AM4. A fila de arcos é inicializada com os dois arcos direcionados da restrição bínária: (CPU, MB) e (MB, CPU).

No **Passo 2**, o arco (CPU, MB) é retirado da fila para verificação. O algoritmo examina cada valor do domínio do Processador e verifica se existe, no domínio da Placa-mãe, ao menos um valor que satisfaça a restrição de soquete. Para o valor AM4, existe suporte no domínio da Placa-mãe. Para o valor AM5, não existe suporte, pois a placa disponível suporta apenas AM4, a incompatibilidade é identificada.

No **Passo 3**, o valor AM5 é removido do domínio do Processador por não possuir suporte na variável vizinha. O arco (MB, CPU) é reinserido na fila para que o algoritmo verifique se as remoções realizadas tornaram algum valor da Placa-mãe inconsistente.

No **Passo 4**, o arco (MB, CPU) é processado. O valor AM4 da Placa-mãe encontra suporte no valor AM4 do Processador, portanto nenhuma remoção adicional é necessária. A fila esvazia e o algoritmo encerra no ponto de estabilidade: ambas as variáveis possuem domínios consistentes com a restrição, sem necessidade de explorar nenhuma atribuição completa por força bruta.

O resultado ilustra o mecanismo central do motor de inferência: uma única seleção do usuário dispara a propagação de restrições por toda a rede, reduzindo os domínios disponíveis de forma incremental e em tempo real (Mackworth, 1977; Russell; Norvig, 2010).

2.7 Sistema Especialista

Um Sistema Especialista (SE) é um programa de computador projetado para simular o raciocínio de um especialista humano em um domínio restrito e bem definido (Russell; Norvig, 2010). Diferentemente de sistema de propósito geral, um SE concentra seu conhecimento em um domínio específico e é capaz de resolver problemas nesse domínio com nível de competência comparável ao de um especialista humano (Todd, 1992). Sua arquitetura é composta por três elementos centrais: a base de conhecimento, que armazena fatos e regras sobre o domínio, o motor de inferência, que aplica essas regras aos fatos disponíveis para derivar conclusões e a interface de explicação, que permite ao sistema justificar suas respostas ao usuário (Russell; Norvig, 2010).

O motor de inferência pode operar segundo encadeamento progressivo (*forward chaining*), partindo dos fatos conhecidos em direção a conclusões, ou encadeamento regressivo (*backward chaining*), partindo de uma hipótese e verificando se os fatos disponíveis a sustentam (Russell; Norvig, 2010). No sistema proposto, o motor opera predominantemente em modo progressivo: a cada seleção de componente pelo usuário, o sistema propaga as restrições a partir desse novo fato para derivar quais opções permanecem válidas.

2.7.1 Explanation Facilities

A capacidade de justificar o próprio raciocínio, denominada na literatura como *explanation facility*, é o que distingue fundamentalmente um SE de um mecanismo de filtragem comum (Clancey, 1983). Segundo (Clancey, 1983), essa capacidade exige que o sistema mantenha não apenas as conclusões derivadas, mas também a cadeia de regras e restrições que levou a cada conclusão, de modo que possa reconstruir e comunicar o raciocínio em linguagem acessível ao usuário. Um mecanismo de filtragem simples responde apenas à pergunta "quais opções são válidas?", apresentando resultados sem elaboração. Um SE com *explanation facility* responde também à pergunta "por que as demais opções são inválidas?", articulando a regra específica que foi violada.

No contexto do sistema proposta, essa capacidade se manifesta da seguinte forma, ao bloquear a seleção de um módulo DDR4 após a escolha de uma CPU AM5, o sistema não se limita a remover o componente da lista de opções, ele apresenta ao usuário uma justificativa explícita como "Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionada suporta exclusivamente DDR5 e este módulo é DDR4". Essa explicação transforma cada bloqueio em uma oportunidade de aprendizado sobre as restrições físicas do *hardware*.

2.7.2 Inteligência Artificial Explicável (XAI)

O problema identificado por Clancey (1983), a necessidade de sistemas inteligentes justificarem seu raciocínio ao usuário, permaneceu latente por décadas e ressurgiu com força renovada no contexto da Inteligência Artificial (IA) moderna sob o nome de XAI (*eXplainable Artificial Intelligence*), ou Inteligência Artificial Explicável. Barredo Arrieta *et al.* (2020) definem XAI como o conjunto de processos e técnicas que permitem que modelos de inteligência artificial produzam resultados compreensíveis por seres humanos, especialmente por usuários finais sem formação técnica em IA. Os autores identificam a explicabilidade como uma necessidade crítica para a adoção prática de sistemas de IA em domínios sensíveis, argumentando que um sistema que apenas produz resultados sem justificá-los não gera confiança suficiente para ser adotado em decisões reais.

A distinção central que o campo de XAI estabelece é entre sistemas caixa-preta (*black-box*), cujo processo de decisão interno é opaco ao usuário, e sistemas transparentes, cujo raciocínio pode ser inspecionado e compreendido (Barredo Arrieta *et al.*, 2020). Sistemas especialistas baseados em CSP pertencem naturalmente à segunda categoria, as variáveis, os domínios e as restrições são explícitos e auditáveis e cada decisão de bloqueio pode ser rastreada até a restrição específica que a originou. Nesse sentido, o motor de inferência CSP/AC-3 proposto neste trabalho é intrinsecamente explicável por *design*, não por técnica de pós-processamento, mas pela própria natureza simbólica do modelo. Isso posiciona o sistema proposto em alinhamento com os princípios contemporâneos de XAI, cada resultado produzido pelo motor de inferência é acompanhado da justificativa técnica que o fundamenta, tornando o processo de validação de *hardware* transparente e auditável pelo usuário.

2.7.3 Mapeamento ao Sistema Proposto

O sistema proposto neste trabalho se enquadra na categoria de sistemas especialistas baseados em restrições. Os três componentes arquiteturais do SE mapeiam diretamente à implementação: a base de conhecimento corresponde à camada de dados que armazena as especificações técnicas dos componentes e as regras de compatibilidade modeladas como restrições do CSP; o motor de inferência corresponde ao algoritmo AC-3 implementado na camada de lógica da aplicação, responsável por propagar restrições e podar domínios a cada interação do usuário; e a interface de explicação corresponde aos alertas técnicos e educativos exibidos na camada de apresentação, que comunicam ao usuário não apenas quais componentes são bloqueados, mas as razões físicas e técnicas subjacentes a cada bloqueio (Todd, 1992).

2.8 Dimensão Educativa em Sistemas Especialistas

O caráter educativo do sistema proposto não decorre da adoção de uma arquitetura de aprendizagem adaptativa, mas da forma como o motor de inferência comunica suas conclusões ao usuário. Para delimitar com precisão essa dimensão, é necessário distinguir três categorias de sistemas que operam sobre domínios de conhecimento restrito: os mecanismos de filtragem, os Sistemas Tutores Inteligentes (STI) e os Sistemas Especialistas com interface educativa.

2.8.1 Mecanismo de Filtragem versus Sistema com Explicação

Um mecanismo de filtragem tradicional opera sobre um conjunto de dados e retorna os elementos que satisfazem determinados critérios, sem qualquer elaboração sobre o processo de decisão. No contexto de compatibilidade de *hardware*, um filtro responde à pergunta "quais componentes são compatíveis com minha seleção atual?", apresentando uma lista de resultados válidos e ocultando ou desabilitando os inválidos, sem indicar o motivo da exclusão. Plataformas comerciais como o *PC Part Picker* operam predominantemente nesse paradigma: informam a incompatibilidade, mas raramente explicam a regra física que a origina.

A limitação pedagógica desse modelo é nítida: o usuário aprende o que não pode fazer, mas não aprende o motivo. Cada bloqueio silencioso é uma oportunidade de aprendizagem perdida (Clancey, 1983). Um sistema que apenas filtra não transfere conhecimento, apenas restringe escolhas.

2.8.2 Sistemas Tutores Inteligentes

Sistemas Tutores Inteligentes (STI) representam o paradigma mais sofisticado de instrução mediada por computador. Sua arquitetura formal é composta por quatro modelos interdependentes: o modelo de domínio, que representa o conhecimento a ser ensinado; o modelo de estudante, que representa o estado de conhecimento individual do aprendiz e é atualizado continuamente com base em seus erros e acertos; o modelo pedagógico, que define as estratégias de instrução e adaptação; e o modelo da interface, que define como o conteúdo é apresentado (Woolf, 2008). O elemento central que distingue um STI de outros sistemas é o modelo de estudante, ele exige que o sistema mantenha um perfil persistente por usuário, rastreando o histórico de interações ao longo de múltiplas sessões para personalizar a instrução (Anderson *et al.*, 1995).

O sistema proposto neste trabalho não implementa a arquitetura de um STI. Não há rastreamento de sessões individuais, não há perfil persistente de usuário e não há adaptação sequencial de conteúdo baseado em histórico. Essa delimitação é intencional, pois o objetivo é criar um sistema especialista com interface educativa, que se concentra na

explicação de cada bloqueio como uma oportunidade de aprendizagem, sem a complexidade adicional de um modelo de estudante adaptativo.

2.8.3 Sistema Especialista Educacional

O sistema proposto se enquadra na categoria de Sistema Especialista Educacional, um SE dotado de *explanation facility* cuja interface é projetado com objetivo pedagógico explícito. A distinção em relação a um SE convencional reside na forma como as conclusões são comunicadas. Enquanto um SE padrão pode simplesmente retornar um resultado binário (compatível ou incompatível), um SE educacional articula a cadeia de raciocínio que levou aquela conclusão em linguagem acessível ao usuário (Clancey, 1983).

No sistema proposto, essa dimensão educativa se manifesta de forma precisa e delimitada: ao detectar uma violação de restrição via propagação AC-3, o sistema não limita a bloquear o componente incompatível, ele apresenta ao usuário uma justificativa explícita que identifica qual restrição foi violada e por qual razão técnica. Por exemplo, ao tentar selecionar um módulo DDR4 após escolher uma CPU com plataforma AM5, o sistema exibe: "Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4". Essa explicação transforma cada bloqueio em uma oportunidade de aprendizagem sobre os princípios físicos que regem a montagem de computadores, sem requerer qualquer cadastro, histórico ou personalização por parte do usuário.

Essa abordagem é pedagogicamente coerente com público-alvo do sistema, o usuário leigo que realiza uma única sessão de configuração, para quem a explicação contextualizada e imediata é mais eficaz do que qualquer mecanismo de adaptação baseado em histórico. O adjetivo "Educacional" no título do trabalho refere-se precisamente a essa capacidade de instruir o usuário sobre as restrições do domínio no exato momento em que elas se tornam relevantes para sua decisão.

2.9 Arquitetura de Software Orientada a Serviços

2.9.1 Arquitetura em Camadas

A arquitetura de *software* compreende o conjunto de decisões estruturais fundamentais que definem a organização de um sistema, as responsabilidades de seus componentes e as interfaces através das quais eles se comunicam (Fowler, 2002). Um dos padrões arquiteturais mais consolidados na engenharia de *software* é a arquitetura em camadas, que organiza o sistema em níveis hierárquicos de abstração, cada um com responsabilidades bem delimitadas e comunicando-se apenas com a camada imediatamente adjacente (Fowler, 2002).

As três camadas clássicas são: a camada de apresentação, responsável pela interação com o usuário e pela exibição de informações, a camada de lógica de negócio, responsável pelo processamento, pelas regras do domínio e pelas decisões da aplicação e a camada de dados, responsável pelo armazenamento, recuperação e persistência de informações (Fowler, 2002). Essa separação aumenta a manutenibilidade do sistema, alterações em uma camada não propagam mudanças para as demais, e facilita a testabilidade, permitindo que cada camada seja verificada de forma independente.

2.9.2 REST como Estilo Arquitetural

A comunicação entre camadas distribuídas em sistemas *web* segue estilos arquiteturais que impõem restrições sobre como os componentes interagem. O estilo mais amplamente adotado é o REST (*Representational State Transfer*), definido por (Fielding, 2000) em sua tese de doutorado como um conjunto de seis princípios arquiteturais para sistemas distribuídos baseados em rede.

Os princípios mais relevantes para o sistema proposto são:

O princípio de cliente-servidor, que estabelece a separação entre a interface do usuário e o armazenamento e processamento de dados, permitindo que cada lado evolua de forma independente. No sistema proposto, o cliente e o servidor evoluem independentemente, a interface pode ser redesenhada sem alterar o motor CSP, e o motor CSP pode ser otimizado sem alterar a interface.

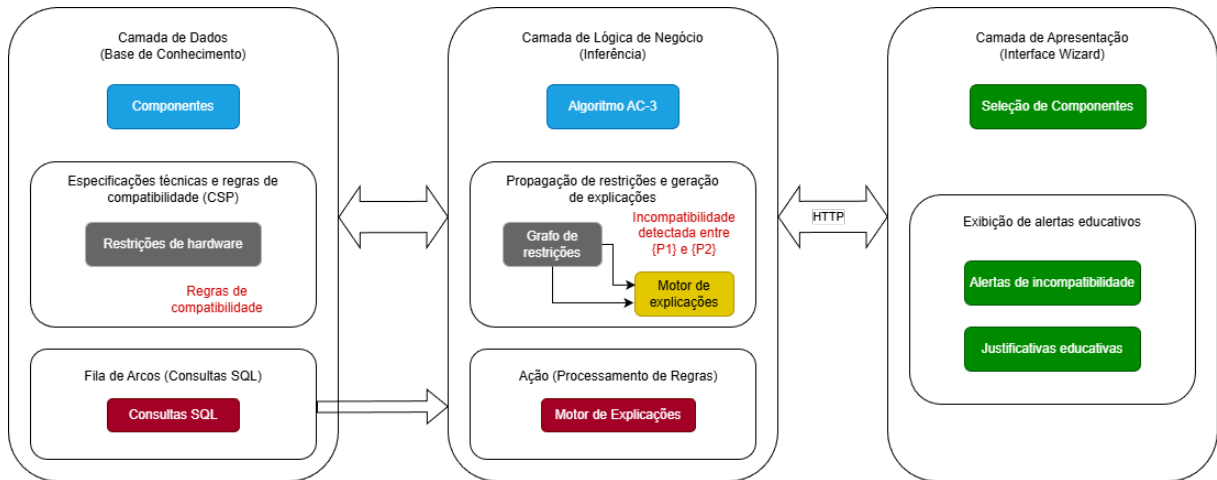
O princípio *stateless* (sem estado), que estabelece que cada requisição enviado pelo cliente ao servidor deve conter todas as informações necessárias para ser compreendida e processada, sem depender de contexto ou sessão armazenada no servidor entre requisições (Fielding, 2000). No contexto do sistema proposto, isso significa que cada chamada à *API* carrega o estado atual completo das seleções do usuário, quais componentes foram escolhidos e quais permanecem em aberto e o motor CSP executa a propagação de restrições a partir desse estado, sem manter sessão persistente no servidor. Essa característica garante escalabilidade e simplicidade operacional.

O princípio da interface uniforme, que estabelece que a comunicação entre o cliente e servidor deve seguir um contrato padronizado, baseado em recursos identificados por URIs e operações semânticas bem definidas (Richardson; Ruby, 2007). No sistema proposto, cada categoria de componente e cada conjunto de restrições é exposto como um recurso REST, acessível por operações HTTP padronizadas.

2.9.3 Mapeamento ao Sistema Proposto

A arquitetura do sistema proposto realiza concretamente os princípios da arquitetura em camadas e do estilo REST, conforme ilustrado na Figura ??.

Figura 3 – Diagrama da arquitetura em três camadas do sistema



Fonte: Elaborada pelo autor

A camada de dados, representada no painel esquerdo da figura, armazena a base de conhecimento do sistema especialista: os modelos de componentes de *hardware*, as especificações técnicas que constituem as variáveis do CSP, as restrições de compatibilidade que compõem o conjunto C , e as regras de compatibilidade que fundamentam as decisões do motor de inferência. O acesso a essa camada é realizado por meio de consultas SQL disparadas pela camada de lógica de negócio.

A camada de lógica de negócio, representada no painel central, é o núcleo inteligente do sistema. Após receber uma requisição via *REST API* contendo o estado atual das seleções do usuário, essa camada executa o Algoritmo AC-3 sobre o grafo de restrições: identifica os arcos inconsistentes, poda os domínios das variáveis afetadas e, ao detectar uma incompatibilidade entre dois componentes P_1 e P_2 , aciona o motor de explicações, que formula a justificativa técnica a ser apresentada ao usuário. O resultado do processamento é retornado à camada de apresentação como resposta à requisição REST.

A camada de apresentação, representada no painel direito, é a interface *wizard* com a qual o usuário interage diretamente. Ela é responsável por exibir o catálogo de componentes disponíveis para seleção, apresentar os alertas de incompatibilidade gerados pelo motor de inferência e comunicar as justificativas educativas que fundamentam cada bloqueio, transformando o resultado da validação CSP em uma oportunidade de aprendizagem sobre os princípios físicos que regem a montagem de computadores.

A separação entre as três camadas garante que o motor de inferência possa ser mantido e expandido de forma independente da interface de apresentação, preservando a modularidade arquitetural ao longo do ciclo de desenvolvimento do sistema (Fowler, 2002; Fielding, 2000).

2.10 Trabalhos Relacionados

Esta seção mapeia o estado da arte em duas frentes: as plataformas comerciais de verificação de compatibilidade de *hardware* atualmente disponíveis e os trabalhos acadêmicos sobre sistemas especialistas e CSP aplicados a domínios similares. O objetivo é evidenciar as lacunas existentes nas soluções atuais e posicionar a contribuição do presente trabalho em relação a elas.

2.10.1 Plataformas Comerciais de Compatibilidade de Hardware

O PC Part Picker é a plataforma de referência global para montagem de computadores. Permite ao usuário selecionar componentes de um catálogo extenso e verifica automaticamente incompatibilidades entre as peças escolhidas (PCPartPicker LLC, 2024). Quando uma incompatibilidade é detectada, a plataforma exibe um alerta genérico indicando que os componentes são incompatíveis, mas sem detalhar a regra técnica violada, por exemplo, ao selecionar uma CPU AM5 com um módulo DDR4, o sistema indica o conflito sem explicar a diferença entre os padrões de memória ou a razão física da incompatibilidade. Do ponto de vista da teoria de sistemas especialistas, o PC Part Picker implementa um mecanismo de filtragem sem *explanation facility*, o usuário recebe o resultado da validação, mas não o raciocínio que o produziu (Clancey, 1983).

O Meupc.net é a alternativa nacional mais utilizada no contexto brasileiro, operando de forma similar ao PC Part Picker com catálogo voltado ao mercado local (Meupc.net, 2024). Apresenta as mesmas limitações em termos de explicação, as incompatibilidades são sinalizadas visualmente, mas sem justificativa técnica que permita ao usuário compreender o princípio de *hardware* subjacente à restrição.

A limitação comum a ambas as plataformas é estrutural, por operarem como mecanismos de filtragem e não como sistemas especialistas com capacidade explicativa, não transferem conhecimento ao usuário, apenas restringem escolhas. Um usuário que utiliza PC Part Picker repetidamente aprende quais componentes são compatíveis entre si, mas não aprende por que são compatíveis, o que o impede de generalizar o conhecimento adquirido para novas situações de compra.

2.10.2 Trabalhos Acadêmicos Relacionados

Na literatura acadêmica, a aplicação de CSPs a problemas de configuração de produtos é um tema consolidado. Sabin e Weigel (1998) apresentam uma *survey* abrangente sobre sistemas de configuração baseados em CSP, identificando a modelagem de dependências entre componentes como variáveis e restrições é a abordagem mais expressiva para domínios com regras de compatibilidade complexas. Os autores demonstram que sistemas de configuração baseada em CSP superam abordagem baseadas em regras estáticas tanto

em expressividade quanto em eficiência de manutenção de base de conhecimento, resultado diretamente aplicável ao domínio de *hardware* proposto neste trabalho.

Mittal e Frayman (1989) introduzem o conceito de *dynamic CSP*, no qual o conjunto de variáveis e restrições pode ser alterado durante a busca, uma extensão relevante para sistemas de configuração interativos onde o usuário faz escolhas incrementais, como é o caso do sistema proposto. Essa abordagem sustenta a decisão arquitetural de propagar restrições a cada interação do usuário, em vez de processar o CSP completo apenas ao final da configuração.

No campo de sistemas especialistas educacionais, Nwana (1990) discute os requisitos para que um sistema especialista seja considerado efetivamente educativo, identificando a capacidade de explicação como condição necessária, mas não suficiente, para a transferência de conhecimento. O autor distingue sistemas que explicam o que de sistemas *o que* explicam *por que*, argumentando que apenas os últimos têm impacto pedagógico real. Esse referencial sustenta diretamente a decisão de projeto de exibir não apenas o resultado da validação, mas a regra de compatibilidade violada em linguagem acessível ao usuário.

Mais recentemente, Wang e Wu (2018) desenvolveram um sistema especialista educacional para apoio ao aprendizado de classificação botânica em ambiente móvel, demonstrando que sistemas especialistas com *feedback* explicativo produzem ganhos mensuráveis de aprendizagem em comparação com sistemas que apenas apresentam informação estática. O estudo é relevante para o presente trabalho por demonstrar empiricamente que a capacidade de explicação, e não apenas a apresentação de conteúdo, é o fator determinante para o impacto pedagógico de um sistema especialista, validando a abordagem adotada no sistema de compatibilidade de *hardware* proposto.

2.10.3 Posicionamento do Trabalho Proposto

A análise das plataformas comerciais e dos trabalhos acadêmicos permite identificar com precisão a contribuição do presente trabalho. Em relação as plataformas comerciais, o diferencial está na *explanation facility*. enquanto PC Part Picker e Meupc.net sinalizam incompatibilidades sem explicá-las, o sistema proposto articula a regra técnica violada em cada bloqueio, transformando a interação em uma oportunidade de aprendizagem. Em relação aos trabalhos acadêmicos, a contribuição está na combinação de três elementos que, individualmente, já são estudados na literatura, CSP como motor de inferência (Sabin; Weigel, 1998), sistema especialista com capacidade explicativa alinhada a princípios de XAI (Clancey, 1983; Barredo Arrieta *et al.*, 2020) e interface educativa com impacto pedagógico demonstrável (Wang; Wu, 2018), em um sistema *web* completo validado sobre um domínio concreto e de relevância prática imediata para o usuário.

3 MATERIAIS E MÉTODOS

De acordo com a natureza e os objetivos do presente trabalho, foram adotadas as seguintes metodologias de pesquisa para embasar o desenvolvimento do sistema especialista educacional proposto.

3.0.1 Bibliográfica

Durante o desenvolvimento deste trabalho, aplica-se a metodologia bibliográfica, por meio da qual foi realizada a análise de materiais já publicados. Para isso, fez-se necessária a avaliação e fundamentação sobre os conteúdos consultados podendo eles ser "de livros, periódicos, documentos, textos, mapas, fotos, manuscritos e, até mesmo, de material disponibilizado na internet etc" (Fontelles *et al.*, 2009).

Em primeira instância, a pesquisa foi conduzida com a definição dos temas e objetivos necessários, com o intuito de identificar os assuntos que sustentam a narrativa do trabalho, esclarecendo quais pontos são úteis e relevantes para a pesquisa. Os temas centrais investigados foram: Problemas de Satisfação de Restrições (CSP), algoritmos de propagação de restrições, sistemas especialistas, inteligência artificial explicável (XAI), sistemas tutores inteligentes e arquitetura de software orientada a serviços. Além disso, foi necessário delimitar um período para a seleção das fontes, priorizando materiais mais recentes, sem desconsiderar obras clássicas e fundamentais para o domínio, como os trabalhos de Clancey (1983), Mackworth (1977) e Russell e Norvig (2010), dada a relevância teórica permanente de tais referências.

Após essa etapa inicial, foram selecionadas como principais bases de busca o Google Scholar e o Portal de Periódicos da CAPES, acessado por meio do sistema CAFe (Comunidade Acadêmica Federada), em razão de sua vasta cobertura de artigos científicos, livros, teses, dissertações e demais produções acadêmicas indexadas. Foram também realizadas buscas direcionadas em portais de conferências e periódicos especializados em inteligência artificial, engenharia de *software* e sistemas de informação.

Dado o volume de resultados retornados, realizou-se uma triagem dos materiais recuperados por meio da leitura de resumos. Os materiais considerados relevantes foram incluídos, nos casos de dúvidas quando à permanência, procedeu-se a uma leitura mais minuciosa. Após a seleção, foi realizada uma análise detalhada dos materiais e a síntese das informações coletadas.

Por fim, todas as pesquisas e extrações de informações foram revisadas, de modo a evitar erros e má interpretação. A revisão foi conduzida em duas etapas, com releitura do

material, da síntese e da extração de informação, resultando em ajustes quando necessário e garantindo a qualidade e a precisão das informações utilizadas ao longo do trabalho.

3.0.2 Aplicada

A pesquisa aplicada orienta o desenvolvimento prático de compatibilidade, cujo produto é um sistema especialista educacional para validação de compatibilidade de *hardware*. Segundo Gerhardt e Silveira (2009), a pesquisa aplicada tem como objetivo gerar conhecimento para aplicação prática e dirigidos à solução de problemas específicos. Neste contexto, os fundamentos teóricos levantados pela pesquisa bibliográfica, notadamente o formalismo de CSP, o algoritmo AC-3 e os princípios de sistemas especialistas com *explanation facility*, foram diretamente empregados na concepção e implementação do sistema proposto.

Essa abordagem permitiu que as decisões arquiteturais e de projeto fossem fundamentadas em evidências da literatura, como a adoção da arquitetura em três camadas (Fowler, 2002), o estilo REST para comunicação entre camadas (Fielding, 2000) e o algoritmo AC-3 como motor de inferência (Mackworth, 1977), garantindo o alinhamento entre os resultados práticos obtidos e o referencial teórico consolidado.

3.1 Materiais

3.1.1 Equipamentos

Para a realização do desenvolvimento, codificação e testes do sistema, foi utilizado um computador com processador Ryzen 7 5700X, 32GB de memória RAM, placa de vídeo AMD Radeon RX 7600 e sistema operacional Windows 11. Esses recursos foram suficientes para executar o ambiente de desenvolvimento, o servidor de aplicação, o sistema de gerenciador de banco de dados e os navegadores necessários para a validação da interface.

3.1.2 Softwares

Para o desenvolvimento do sistema foram utilizadas ferramentas de *software* selecionadas com base em sua adequação técnica ao projeto, disponibilidade gratuita ou de código aberto e familiaridade prévia do desenvolvedor.

3.2 Ferramentas Utilizadas

3.2.1 Visual Studio Code

O Visual Studio Code (VS Code) é um editor de código-fonte desenvolvido pela Microsoft, disponível gratuitamente para sistemas operacionais Windows, Linux e MacOS. Por meio de extensões, a ferramenta agrega funcionalidades como depuração, controle de versão, terminal integrado e suporte a diversas linguagens de programação, aproximando-se de um ambiente de desenvolvimento integrado (IDE).

No presente trabalho, o VS Code foi utilizado como ambiente principal de desenvolvimento, tanto para codificação do *backend* e do *frontend* quanto para a edição e compilação do documento LaTeX deste trabalho, por meio da extensão LaTeX Workshop. A escolha pela ferramenta deve-se à sua leveza, à vasta disponibilidade de extensões relevantes para o ecossistema adotado e ao conhecimento prévio do desenvolvedor.

3.2.2 Node.js e NestJS

O Node.js é um ambiente de execução JavaScript de código aberto e multiplataforma, construído sobre o motor V8 do Google Chrome, que permite a execução de código JavaScript no lado do servidor, sendo amplamente empregado no desenvolvimento de aplicações *web* escaláveis e de alta performance. O NestJS é um *framework* para desenvolvimento de aplicações server-side organizada em módulos, *controller* e *services*. Essa estrutura favorece a separação de responsabilidades e a manutenibilidade do código, alinhando-se diretamente aos princípios da arquitetura em camadas descrito por Fowler (2002).

No contexto do sistema proposto, o Node.js com NestJS compõe a camada de lógica de negócio, sendo o responsável por receber as requisições da interface, executar o algoritmo AC-3 sobre o grafo de restrições de *hardware* e retornar ao cliente os resultados da validação acompanhados das justificativas educativas correspondentes.

3.2.3 PostgreSQL e Docker

O PostgreSQL é um banco de dados relacional de código aberto, com suporte a transações, multiplataforma e amplamente utilizado em aplicações *web* de diferentes escalas. A escolha por esse sistema deve-se à sua robustez, à sua conformidade com o padrão SQL e à facilidade de integração com ecossistemas Node.js.

No sistema proposto, o PostgreSQL é utilizado para armazenar a base de conhecimento do sistema especialista: os modelos de componentes de *hardware*, suas especificações técnicas e as regras de compatibilidade que compõe o conjunto de restrições do CSP.

Essas informações são consultadas pela camada de lógica de negócio a cada execução do algoritmo AC-3.

O provisionamento do banco de dados foi realizado por meio do Docker, uma plataforma de código aberto para criação e execução de contêineres de *software*. A utilização do Docker permite que o ambiente do banco de dados seja definido de forma declarativa e reproduzível, garantindo consistência entre os ambientes de desenvolvimento e produção sem a necessidade de instalação direta do PostgreSQL no sistema operacional.

3.2.4 DBeaver

O DBeaver é uma ferramenta de administração de banco de dados de código aberto e multiplataforma, compatível com diversos sistemas gerenciadores, incluindo o PostgreSQL. A ferramenta oferece uma interface visual para a execução de consultas SQL, navegação entre tabelas, modelagem de dados e exportação de resultados.

No presente trabalho, o DBeaver foi utilizado para o auxílio na administração e manipulação do banco de dados durante o desenvolvimento, permitindo a inspeção e validação dos dados cadastrados na base de conhecimento do sistema especialista.

3.2.5 Next.js

O Next.js é um *framework* de código aberto para o desenvolvimento de aplicações *web* contruído sobre o React, desenvolvido e mantido pela Vercel. O *framework* oferece recursos como renderização híbrida, roteamento baseado em sistemas de arquivos e suporte nativo a APIs, facilitando o desenvolvimento de aplicações *web* com uma única base de código.

No presente trabalho, o Next.js foi utilizado para o desenvolvimento da camada de apresentação do sistema, implementada como um interface no formato *wizard* que guia o usuário, passo a passo, na seleção dos componentes de *hardware*. A escolha do Next.js deve-se à sua capacidade e integrar a interface e as chamadas à camada de lógica de negócio em um único projeto, além de atualizar dinamicamente os componentes exibidos na tela em resposta ao resultado das validações retornadas pelo motor de inferência.

3.2.6 Figma

O Figma é uma ferramenta de *design* de interface baseada em nuvem, acessada por meio de navegador *web*, que permite a criação de protótipos e *layouts* de forma colaborativa em tempo real, durante o processo de criação é possível fazer uso de diferente modelos e ter uma visibilidade geral do projeto.

No presente trabalho, o Figma foi utilizado para o desenvolvimento dos protótipos das interfaces do sistema, permitindo planejar e validar o *layout* da interface *wizard* antes da sua implementação na camada de apresentação. A escolha pela ferramenta deve-se à sua disponibilidade gratuita para uso acadêmico e ao conhecimento prévio do desenvolvedor.

3.2.7 Astah Community

O Astah Community é uma ferramenta de modelam UML desenvolvida pela empresa japonesa Change Vision, que possibilita a criação de diagramas de casos de uso, de classes, de sequência, de domínio e de atividades, entre outros. A ferramenta disponibiliza uma versão gratuita voltada ao uso acadêmico.

No presente trabalho, o Astah Community foi utilizado para a elaboração dos diagramas UML, que compõe as etapas de análise e projeto de *software*, incluindo o diagrama de casos de uso, de domínio, de classes e de sequência (por enquanto).

3.2.8 Teste?

3.3 Métodos de Desenvolvimento

3.3.1 Modelagem do Problema como CSP

A etapa central do método de desenvolvimento consistiu na formalização do problema de compatibilidade de *hardware* como um Problema de Satisfação de Restrições (CSP). Conforme descrito por Russell e Norvig (2010), um CSP é caracterizado por uma tripla (X, D, C) , na qual X representa o conjunto de variáveis, D os respectivos domínios e C o conjunto de restrições.

No sistema proposto, cada categoria de componente de *hardware* foi mapeada a uma variável do CSP, cujo domínio é composto pelos modelos comercialmente disponíveis cadastrados na base de conhecimento. As restrições foram identificadas a partir das especificações técnicas oficiais dos fabricantes e formalizadas como restrições binárias entre pares de variáveis.

3.3.2 Implementação do Algoritmo AC-3

O motor de inferência do sistema foi implementado com base no algoritmo AC-3 (*Arc Consistency Algorithm 3*), proposto por Mackworth (1977). A implementação seguiu a estrutura algorítmica descrita por Russell e Norvig (2010), uma fila de arcos é inicializada com todos os pares de variáveis relacionadas por restrições binárias, para cada arco retirado da fila, o algoritmo verifica se todos os valores do domínio da variável de origem possuem

suporte no domínio da variável de destino, valores sem suporte são removidos do domínio e, a cada remoção, os arcos afetados são reinseridos na fila para propagação.

No contexto do sistema proposto, o AC-3 é invocado a cada seleção de componente realizada pelo usuário na interface. O estado atual das seleções é enviado à camada de lógica de negócio, implementada com NestJS, que executa a propagação de restrições e retorna ao cliente os domínios atualizados, juntamente com as justificativas técnicas referentes a cada valor removido. Essa abordagem garante que a validação ocorra de forma incremental e em tempo real, tornando viável a execução sobre catálogos de tamanho realista sem ocorrer explosão combinatória, o que aconteceria no caso de validação por força bruta.

3.3.3 Implementação das Justificativas Educativas

Para cada restrição violada pelo AC-3, o sistema aciona um módulo de geração de justificativas, responsável por formular, em linguagem natural, a explicação técnica correspondente à incompatibilidade identificada. Esse módulo implementa o conceito de *explanation facility* descrito por Clancey (1983), que distingue um sistema especialista de um mecanismo de filtragem simples pela sua capacidade de articular a cadeia de regras e restrições que levou a uma determinada conclusão.

As justificativas foram estruturadas de forma a identificar explicitamente qual restrição foi violada, quais componentes envolvidos e qual o princípio técnico subjacente à incompatibilidade. Por exemplo, ao detectar a seleção simultânea de um processador com plataforma AM5 e um módulo de memória DDR4, o sistema exibe ao usuário a seguinte justificativa: *"Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4"*. Dessa forma, cada bloqueio é transformado em uma oportunidade de aprendizagem sobre as restrições físicas que regem a montagem de computadores.

3.3.4 Processo de Desenvolvimento de Software

O desenvolvimento do sistema seguiu uma abordagem iterativa e incremental, na qual as funcionalidades foram implementadas e validadas em etapas sucessivas. Em cada iteração, foram realizadas as atividades de análise de requisitos, projeto, implementação e teste, permitindo a identificação e correção precoce de inconsistências entre os requisitos especificados e o comportamento do sistema.

A organização das tarefas foi realizada por meio de uma ferramenta de gerenciamento de projetos, com atividades distribuídas conforme seu estado de desenvolvimento: a fazer, em desenvolvimento, em teste, concluído e adiado. Essa estrutura permitiu o acompanhamento contínuo do progresso do projeto e a priorização das atividades ao longo

do ciclo de desenvolvimento.

3.3.5 Estratégia de Testes

4 DESCRIÇÃO GERAL DO PRODUTO

Este capítulo apresenta uma visão geral do produto desenvolvido neste trabalho, descrevendo sua finalidade, seu público-alvo, suas principais funcionalidades e as características que o distinguem das soluções existentes. O detalhamento dos requisitos, da modelagem e da implementação é apresentado nos capítulos seguintes.

O produto é um sistema especialista educacional para validação de compatibilidade de hardware, disponibilizado como uma aplicação web. Sua finalidade é auxiliar o usuário na montagem de um computador desktop, verificando em tempo real se os componentes selecionados são compatíveis entre si e, sempre que uma incompatibilidade é detectada, explicando em linguagem acessível a razão técnica do bloqueio. O sistema não se limita a apontar que dois componentes não funcionam juntos, mas comunica qual regra física foi violada, transformando cada verificação em uma oportunidade de aprendizagem sobre os princípios que regem a montagem de computadores.

O público-alvo principal é o usuário leigo ou iniciante, aquele que deseja montar o próprio computador mas não domina as especificações técnicas que determinam a compatibilidade entre processador, placa-mãe, memória, placa de vídeo e fonte de alimentação. Para esse usuário, um erro de seleção representa prejuízo financeiro e frustração, uma vez que componentes incompatíveis simplesmente não operam em conjunto. O sistema atua de forma preventiva, evitando o erro antes da compra e, ao mesmo tempo, ensinando o usuário a compreender as restrições envolvidas.

4.1 Funcionalidades Principais

A interação com o sistema ocorre por meio de uma interface no formato *wizard*, que conduz o usuário passo a passo pela seleção de cada categoria de componente. A cada escolha, o sistema valida automaticamente a configuração e atualiza a lista de opções disponíveis, sinalizando os componentes que se tornaram incompatíveis com as seleções já realizadas.

Diferentemente das plataformas que ocultam as opções incompatíveis, o sistema mantém esses componentes visíveis e acompanhados de uma justificativa técnica. Assim, ao tentar combinar um processador de soquete AM5 com uma placa-mãe de soquete AM4, o usuário não apenas é impedido de prosseguir, mas recebe a explicação de que os dois soquetes são fisicamente incompatíveis. Essa abordagem preserva o caráter educativo da ferramenta, pois o motivo do bloqueio permanece disponível para consulta.

Ao final do processo, o sistema apresenta um resumo da configuração montada,

indicando seu estado geral de compatibilidade, o consumo energético estimado e o tempo de execução da validação. O usuário pode, a qualquer momento, retroceder para substituir um componente selecionado em etapa anterior ou reiniciar completamente a configuração, sendo a validação reexecutada a cada alteração.

4.2 Diferencial do Produto

O diferencial do produto está na articulação entre duas capacidades que, nas soluções tradicionais, costumam aparecer isoladas. A primeira é a validação formal, realizada por um motor de inferência baseado no formalismo de Problemas de Satisfação de Restrições e no algoritmo AC-3, que propaga as restrições entre os componentes de forma incremental e em tempo real. A segunda é a capacidade de explicação, herdada da teoria dos sistemas especialistas, que fundamenta cada bloqueio com uma justificativa compreensível ao usuário leigo.

As plataformas comerciais de montagem de computadores realizam a validação, mas operam como mecanismos de filtragem, apenas sinalizando conflitos sem esclarecer suas causas. O produto desenvolvido neste trabalho preenche essa lacuna ao unir a verificação técnica à comunicação didática, posicionando-se menos como um filtro de compras e mais como um tutor que orienta o usuário durante a montagem.

Cabe destacar ainda que, embora aplicado ao domínio de compatibilidade de hardware, o motor de inferência do sistema foi projetado de forma genérica, dissociado das regras específicas desse domínio. Todo o conhecimento sobre componentes e restrições reside em uma base de conhecimento configurável, e não no código, o que torna a arquitetura reaproveitável, em princípio, para outros problemas de configuração modeláveis como restrições. O domínio de hardware é, nesse sentido, o estudo de caso escolhido para materializar e validar uma solução de propósito mais amplo.

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE

Este capítulo apresenta o processo de elicitação e análise dos requisitos do sistema especialista educacional para validação de compatibilidade de *hardware*. Inicialmente, são descritos os requisitos do usuário e do sistema, classificados em funcionais e não funcionais. Em seguida, são apresentados os artefatos de análise: casos de uso, modelo de domínio e diagrama de classes de análise, que descrevem a estrutura e o comportamento esperados da aplicação.

5.1 Requisitos do Usuário

Os requisitos do usuário expressam, em linguagem natural, as necessidades que motivam o desenvolvimento do sistema do ponto de vista de quem o utiliza. O usuário-alvo deste trabalho é o consumidor leigo que pretende montar um computador *desktop* e necessita verificar a compatibilidade entre os componentes selecionados.

Nesse contexto, o sistema deve permitir que o usuário selecione componentes de *hardware* de forma guiada e, a cada seleção, seja informado de imediato sobre quais componentes se tornam incompatíveis e, sobretudo, sobre a razão técnica de cada incompatibilidade. Diferentemente das plataformas comerciais existentes, que apenas sinalizam o conflito, o sistema proposto deve transformar cada bloqueio em uma oportunidade de aprendizagem, comunicando ao usuário a regra física ou lógica violada em linguagem acessível.

5.2 Requisitos do Sistema

Os requisitos do sistema traduzem as necessidades do usuário em especificações detalhadas que orientam o projeto e a implementação. Segundo Sommerville (2011), os requisitos de sistema são descrições mais detalhadas das funções, dos serviços e das restrições operacionais do *software*, servindo de contrato entre o que se espera e o que será construído. Assim, esta seção apresenta os requisitos identificados neste trabalho, organizados em funcionais e não funcionais.

5.2.1 Requisitos Funcionais

De acordo com Sommerville (2011), os requisitos funcionais descrevem o que o sistema deve fazer, isto é, os serviços que ele deve fornecer, como deve reagir a determinadas entradas e como deve se comportar em situações específicas.

Os atores que interagem com o sistema são: usuário e sistema. Os requisitos funcionais identificados estão apresentados nos Quadros 1 a 19.

Os requisitos estão classificados quanto à prioridade (*Essencial*, *Importante* ou *Desejável*) e quanto à operação (*Entrada*, *Processamento* ou *Saída*).

Quadro 1 – RF-01 – Listar componentes por categoria

Código	RF-01
Grupo	Catálogo de Componentes
Descrição	Ação: Listar. Objeto: Componentes de <i>hardware</i> agrupados por categoria. Atributos: categoria (variável do CSP), nome do modelo, características associadas à categoria e seus respectivos valores. Exemplos: Usuário acessa o sistema e visualiza a lista de componentes da primeira categoria cadastrada (ex.: CPU); usuário avança e visualiza a lista de componentes da categoria seguinte. Restrições: 1. O catálogo deve ser exibido separado por categoria. 2. As categorias exibidas devem ser obtidas dinamicamente da base de conhecimento, não fixadas no código. 3. Cada componente deve exibir as características associadas à sua categoria, relevantes à compatibilidade.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 2 – RF-02 – Navegar pela interface *wizard*

Código	RF-02
Grupo	Catálogo de Componentes
Descrição	Ação: Navegar. Objeto: Interface <i>wizard</i> de configuração passo a passo. Atributos: etapa atual, categoria de componente em seleção. Exemplos: Usuário conclui a seleção da CPU e avança para a etapa de Placa-mãe; usuário retorna à etapa anterior para revisar a seleção. Restrições: 1. A interface deve guiar o usuário sequencialmente por cada categoria de <i>hardware</i>. 2. A ordem das etapas deve ser determinada pela ordenação das categorias cadastradas na base de conhecimento. 3. O usuário deve poder avançar e retroceder entre etapas.
Prioridade	Essencial
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 3 – RF-03 – Exibir componentes incompatíveis com sinalização visual

Código	RF-03
Grupo	Catálogo de Componentes
Descrição	Ação: Exibir com sinalização visual de bloqueio. Objeto: Componentes incompatíveis com a seleção atual. Atributos: status de compatibilidade, indicador visual de bloqueio. Exemplos: Após seleção de CPU com soquete AM5, todas as placas-mãe com soquete AM4 permanecem visíveis na lista, porém exibidas com indicador de bloqueio. Restrições: 1. Componentes incompatíveis não devem ser ocultados da listagem. 2. Devem ser mantidos visíveis com indicador visual de bloqueio. 3. O usuário deve poder visualizar o motivo do bloqueio ao interagir com o componente.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 4 – RF-04 – Exibir características técnicas dos componentes

Código	RF-04
Grupo	Catálogo de Componentes
Descrição	Ação: Exibir. Objeto: Características técnicas do componente, derivadas da base de conhecimento. Atributos: pares característica-valor associados à categoria do componente – variáveis conforme a categoria. Exemplos: {Ryzen 5 7600: soquete = AM5, TDP = 65 W}; {Kingston DDR5-5600: padrão = DDR5, capacidade = 16 GB}. Restrições: 1. As características devem ser exibidas para cada componente do catálogo. 2. As características exibidas devem ser determinadas pela categoria do componente, não codificadas por tipo no sistema. 3. A inclusão de uma nova característica não deve exigir alteração de código.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 5 – RF-05 – Executar propagação de restrições via algoritmo AC-3

Código	RF-05
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	Ação: Executar propagação de restrições. Objeto: Algoritmo AC-3 sobre o grafo de restrições do CSP, construído dinamicamente a partir da base de conhecimento. Atributos: estado atual das seleções do usuário, domínios das variáveis (categorias cadastradas), fila de arcos derivada das restrições persistidas. Exemplos: Usuário seleciona uma CPU com soquete AM5; o AC-3 é invocado e remove do domínio da categoria vizinha (Placa-mãe) todos os modelos cujo valor de característica viole a restrição cadastrada. Restrições: 1. O AC-3 deve ser invocado a cada seleção de componente realizada pelo usuário. 2. O grafo de restrições deve ser montado em tempo de execução a partir das restrições cadastradas, sem regras codificadas no motor. 3. O estado completo das seleções deve ser enviado à API REST a cada chamada (arquitetura <i>stateless</i>). 4. O resultado deve retornar os domínios atualizados em tempo real, sem atraso perceptível. 5. A complexidade de pior caso deve ser $O(cd^3)$, conforme definido por Mackworth (1977).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 6 – RF-06 – Validar compatibilidade de soquete CPU–Placa-mãe

Código	RF-06
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	Ação: Validar. Objeto: Compatibilidade de soquete entre CPU e Placa-mãe (instância de restrição de igualdade do conjunto C). Atributos: soquete da CPU, soquete suportado pela placa-mãe. Exemplos: {CPU: AM5, Placa-mãe: AM4} → incompatível – bloqueio acionado; {CPU: AM5, Placa-mãe: AM5} → compatível. Restrições: 1. A restrição é binária e absoluta – não admite adaptação de <i>software</i> ou configuração. 2. O soquete da CPU deve ser idêntico ao soquete suportado pela placa-mãe. 3. Incompatibilidade detectada deve bloquear o componente e acionar o módulo de explicação (RF-11).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 7 – RF-07 – Validar padrão de memória RAM–Placa-mãe

Código	RF-07
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	Ação: Validar. Objeto: Padrão de memória entre módulo RAM e Placa-mãe (instância de restrição de igualdade do conjunto C). Atributos: padrão DDR do módulo RAM (DDR4 ou DDR5), padrão DDR suportado pela placa-mãe. Exemplos: {RAM: DDR4-3200, Placa-mãe: B650 (somente DDR5)} → incompatível; {RAM: DDR5-5600, Placa-mãe: B650} → compatível. Restrições: 1. Os padrões DDR4 e DDR5 são física e eletricamente incompatíveis. 2. O padrão do módulo deve corresponder exatamente ao suportado pela placa-mãe. 3. A restrição é transitiva: depende da placa-mãe, que por sua vez depende da CPU selecionada. 4. Incompatibilidade detectada deve bloquear o componente e acionar o módulo de explicação (RF-12).
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 8 – RF-08 – Validar capacidade energética da fonte (TDP)

Código	RF-08
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	Ação: Validar. Objeto: Capacidade energética da fonte de alimentação em relação ao consumo de cada componente (instâncias de restrição quantitativa do conjunto C). Atributos: TDP da CPU (W), TDP da GPU (W), potência da fonte (W), margem de segurança recomendada (20–30%). Exemplos: {GPU TDP: 170 W, Fonte: 250 W} $\rightarrow$ bloqueado ($170\text{ W} \times 1,25 = 212,5\text{ W}$, porém a fonte deve suportar cada demanda com margem e a configuração completa a excede); {GPU TDP: 170 W, Fonte: 550 W} $\rightarrow$ válido. Restrições: 1. A fonte deve suprir a demanda energética de cada componente de maior consumo (CPU e GPU) individualmente, acrescida de margem de segurança de 20–30%, preservando a natureza binária das restrições do grafo. 2. A restrição é quantitativa (desigualdade), diferentemente das restrições binárias de soquete e memória. 3. Fonte subdimensionada deve ser bloqueada e acionar o módulo de explicação com os valores calculados (RF-13). 4. O consumo agregado da configuração é calculado e exibido ao usuário a título informativo, não constituindo restrição do CSP.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 9 – RF-09 – Propagar restrições incrementalmente a cada seleção

Código	RF-09
Grupo	Motor de Inferência (CSP/AC-3)
Descrição	Ação: Propagar restrições e atualizar domínios. Objeto: Domínios das variáveis do CSP após cada interação do usuário. Atributos: variáveis afetadas, valores removidos dos domínios, arcos reinseridos na fila do AC-3. Exemplos: Remoção do valor AM5 do domínio de CPU provoca reinserção do arco (Placa-mãe, CPU) na fila para reavaliação dos modelos de placa-mãe ainda disponíveis. Restrições: 1. A propagação deve ser incremental a cada interação, nunca por avaliação exaustiva (<i>brute force</i>). 2. O motor deve operar até o ponto de estabilidade – quando nenhuma remoção adicional for possível.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 10 – RF-10 – Exibir justificativa técnica para cada incompatibilidade

Código	RF-10
Grupo	Explicações Educativas (<i>Explanation Facility</i>)
Descrição	Ação: Exibir justificativa técnica em linguagem natural. Objeto: Incompatibilidade detectada pelo AC-3. Atributos: restrição violada, componentes envolvidos, princípio técnico subjacente. Exemplos: “Este módulo de memória é incompatível: a placa-mãe compatível com o processador selecionado suporta exclusivamente DDR5, e este módulo utiliza o padrão DDR4.” Restrições: 1. Cada componente bloqueado deve ter uma justificativa exibida ao usuário. 2. A justificativa deve identificar explicitamente qual restrição do CSP foi violada. 3. A linguagem deve ser acessível ao usuário leigo. 4. Nenhum bloqueio pode ocorrer de forma silenciosa (sem explicação ao usuário).
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 11 – RF-11 – Exibir alerta de incompatibilidade de soquete

Código	RF-11
Grupo	Explicações Educativas (<i>Explanation Facility</i>)
Descrição	Ação: Exibir alerta com explicação. Objeto: Incompatibilidade de soquete entre CPU e Placa-mãe. Atributos: soquete exigido pela CPU selecionada, soquete oferecido pela placa-mãe bloqueada. Exemplos: “Esta placa-mãe utiliza soquete AM4, incompatível com o processador selecionado, que exige soquete AM5.” Restrições: 1. O alerta deve nomear explicitamente os dois soquetes envolvidos (ex.: AM4 e AM5). 2. Deve ser exibido no momento em que o usuário tenta selecionar o componente incompatível.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 12 – RF-12 – Exibir alerta de incompatibilidade de padrão de memória

Código	RF-12
Grupo	Explicações Educativas (<i>Explanation Facility</i>)
Descrição	Ação: Exibir alerta com explicação. Objeto: Incompatibilidade de padrão de memória entre módulo RAM e Placa-mãe. Atributos: padrão DDR suportado pela placa-mãe, padrão DDR do módulo bloqueado. Exemplos: “Este módulo utiliza o padrão DDR4. A placa-mãe selecionada suporta exclusivamente DDR5. Os dois padrões são física e eletricamente incompatíveis e não podem ser utilizados juntos.” Restrições: 1. O alerta deve informar ambos os padrões DDR envolvidos. 2. Deve mencionar a natureza física da incompatibilidade, não apenas a lógica.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 13 – RF-13 – Exibir alerta de fonte de alimentação subdimensionada

Código	RF-13
Grupo	Explicações Educativas (<i>Explanation Facility</i>)
Descrição	Ação: Exibir alerta com explicação e valores calculados. Objeto: Fonte de alimentação com potência insuficiente para um componente. Atributos: consumo do componente (W), potência mínima recomendada com margem (W), potência da fonte bloqueada (W). Exemplos: “Esta fonte de 250 W é insuficiente para a Radeon RX 7900 XT, que exige no mínimo 212,5 W com a margem de segurança de 25%.” Restrições: 1. O alerta deve exibir o consumo do componente e a potência mínima recomendada com a margem aplicada. 2. A margem de segurança utilizada no cálculo deve ser informada ao usuário.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 14 – RF-14 – Reinicializar a configuração

Código	RF-14
Grupo	Gerenciamento da Configuração
Descrição	Ação: Reinicializar. Objeto: Configuração de <i>hardware</i> em andamento. Atributos: seleções realizadas, estado do <i>wizard</i>. Exemplos: Usuário clica em “Reiniciar” na etapa 3; todas as seleções são apagadas e o <i>wizard</i> retorna à Etapa 1 (CPU). Restrições: 1. A reinicialização deve apagar todas as seleções realizadas. 2. O <i>wizard</i> deve retornar ao estado inicial (primeira categoria cadastrada). 3. A operação deve estar disponível em qualquer etapa do <i>wizard</i>.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 15 – RF-15 – Substituir componente selecionado em etapa anterior

Código	RF-15
Grupo	Gerenciamento da Configuração
Descrição	Ação: Substituir. Objeto: Componente já selecionado em etapa anterior do <i>wizard</i>. Atributos: etapa do <i>wizard</i>, componente anteriormente selecionado, novo componente escolhido. Exemplos: Usuário retorna à Etapa 1 e troca a CPU por um modelo diferente; o sistema reexecuta o AC-3 com a nova CPU e atualiza a disponibilidade dos componentes nas etapas seguintes. Restrições: 1. O usuário deve poder retroceder a qualquer etapa anterior do <i>wizard</i>. 2. Ao substituir um componente, o sistema deve reexecutar o AC-3 sobre o novo estado completo. 3. Componentes das etapas subsequentes afetados pela substituição devem ser reavaliados automaticamente.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 16 – RF-16 – Exibir resumo da configuração finalizada

Código	RF-16
Grupo	Gerenciamento da Configuração
Descrição	Ação: Exibir. Objeto: Resumo da configuração de <i>hardware</i> finalizada. Atributos: lista dos componentes selecionados, status geral de compatibilidade, consumo total estimado (W). Exemplos: Ao concluir a última etapa, o sistema exibe: {CPU: Ryzen 5 7600, Placa-mãe: ASUS B650, RAM: Kingston DDR5-5600, GPU: RX 7600, Fonte: 550 W – Status: Configuração compatível}. Restrições: 1. O resumo deve ser exibido ao término da seleção de todas as categorias. 2. Deve incluir o status geral de compatibilidade da configuração montada.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 17 – RF-17 – Armazenar e consultar a base de conhecimento do CSP

Código	RF-17
Grupo	Base de Conhecimento
Descrição	Ação: Armazenar e consultar. Objeto: Modelo genérico que representa as variáveis (X), os domínios (D) e as características dos componentes do CSP. Atributos: categoria (variável), componente (valor de domínio), característica (atributo configurável com tipo associado) e valor da característica por componente. Exemplos: categoria = CPU; componente = Ryzen 5 7600; característica = soquete (tipo texto); valor = AM5. Característica = TDP (tipo inteiro); valor = 65. Restrições: 1. Os dados devem ser persistidos em banco de dados PostgreSQL. 2. As características de cada categoria não devem ser fixadas como campos de classe, mas armazenadas como registros configuráveis. 3. Cada característica deve registrar seu tipo (texto ou inteiro) para orientar a forma de comparação no motor de inferência. 4. A base de conhecimento deve ser consultada pela camada de lógica de negócio a cada execução do AC-3.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 18 – RF-18 – Popular base de conhecimento via *Database Seeding*

Código	RF-18
Grupo	Base de Conhecimento
Descrição	Ação: Popular (<i>Database Seeding</i>). Objeto: Base de conhecimento inicial da prova de conceito (categorias, características, componentes, valores e restrições). Atributos: conjunto inicial de categorias, características com seus tipos, componentes com seus valores e restrições para os cenários de validação. Exemplos: <i>Seed</i> com categorias CPU/Placa-mãe/RAM/GPU/Fonte; características soquete, padrão DDR e TDP; CPUs AM4 e AM5; placas-mãe B450/B550 (AM4, DDR4) e B650 (AM5, DDR5); módulos DDR4 e DDR5; e as restrições de soquete, memória e potência. Restrições: 1. A inserção inicial deve ser realizada via <i>Database Seeding</i> no Prisma ORM. 2. O <i>seed</i> deve popular categorias, características, componentes, valores e restrições de forma consistente com o modelo genérico. 3. O processo de <i>seeding</i> deve ser isolado do desenvolvimento de interface administrativa. 4. Os dados devem cobrir ao menos duas plataformas (AM4 e AM5) para viabilizar os cenários de teste.
Prioridade	Desejável
Operação	Entrada
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 19 – RF-19 – Armazenar e consultar o conjunto de restrições (*C*)

Código	RF-19
Grupo	Base de Conhecimento
Descrição	Ação: Armazenar e consultar. Objeto: Conjunto de restrições <i>C</i> do CSP, modelado como registros configuráveis. Atributos: característica de origem, característica de destino, operador (igualdade ou desigualdade), parâmetro opcional (ex.: margem de segurança) e <i>template</i> de justificativa. Exemplos: restrição de soquete = {origem: soquete (CPU), destino: soquete suportado (Placa-mãe), operador: IGUAL}; restrição de potência = {origem: TDP, destino: potência (Fonte), operador: MAIOR_OU_IGUAL, parâmetro: 1,25}. Restrições: 1. Cada restrição deve associar duas características por meio de um operador, definindo uma aresta do grafo de restrições. 2. O parâmetro deve ser utilizado apenas em restrições quantitativas; em restrições de igualdade permanece nulo. 3. Cada restrição deve armazenar o <i>template</i> da justificativa educativa a ser exibida quando for violada. 4. Novas restrições devem poder ser cadastradas sem alteração de código, sendo o grafo construído dinamicamente a partir desses registros.
Prioridade	Essencial
Operação	Processamento
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.2 Requisitos Não Funcionais

Segundo Sommerville (2011), os requisitos não funcionais são restrições sobre os serviços ou funções oferecidos pelo sistema, relacionando-se a propriedades como desempenho, confiabilidade, usabilidade e manutenibilidade. Esta seção apresenta os requisitos não funcionais identificados neste trabalho, apresentados nos Quadros 20 a 33.

Quadro 20 – RNF-01 – Tempo de resposta da validação

Código	RNF-01
Grupo	Desempenho
Descrição	A execução do algoritmo AC-3 e o retorno dos domínios atualizados via API REST devem ocorrer em milissegundos, garantindo validação em tempo real sem degradação perceptível da interface. Restrições: 1. O tempo de resposta da API de validação deve ser inferior a 500 ms para o catálogo da prova de conceito. 2. A resposta deve ser percebida como instantânea pelo usuário durante a interação com o <i>wizard</i>.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 21 – RNF-02 – Ausência de avaliação por força bruta

Código	RNF-02
Grupo	Desempenho
Descrição	O motor de inferência deve operar exclusivamente por propagação de restrições (AC-3), nunca por enumeração exaustiva de combinações, garantindo viabilidade sobre catálogos de tamanho realista. Restrições: 1. A complexidade de pior caso do motor deve ser $O(cd^3)$ – nunca $O(d^n)$. 2. A abordagem por força bruta é proibida como estratégia de validação.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 22 – RNF-03 – Linguagem natural nas explicações de incompatibilidade

Código	RNF-03
Grupo	Usabilidade
Descrição	Todas as justificativas de bloqueio exibidas ao usuário devem ser redigidas em linguagem natural e acessível, sem exigir conhecimento técnico prévio de <i>hardware</i> para sua compreensão. Restrições: 1. Termos técnicos utilizados nas explicações devem ser acompanhados de contexto suficiente para compreensão. 2. A explicação deve responder claramente à pergunta “por que este componente está bloqueado?”.
Prioridade	Essencial
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 23 – RNF-04 – Interface responsiva

Código	RNF-04
Grupo	Usabilidade
Descrição	A camada de apresentação desenvolvida em Next.js deve ser responsiva, funcionando corretamente em navegadores <i>desktop</i> e dispositivos móveis modernos sem perda de funcionalidade. Restrições: 1. O <i>layout</i> deve adaptar-se a diferentes resoluções de tela. 2. Todas as funcionalidades do <i>wizard</i> devem estar acessíveis em dispositivos móveis.
Prioridade	Importante
Operação	Saída
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 24 – RNF-05 – Acesso sem cadastro ou autenticação

Código	RNF-05
Grupo	Usabilidade
Descrição	O sistema deve ser acessível sem criação de conta, <i>login</i> ou qualquer forma de identificação prévia, eliminando barreiras de entrada para o usuário leigo. Restrições: 1. Não deve existir tela de <i>login</i> ou cadastro para o fluxo principal de configuração. 2. Nenhuma sessão persistente de usuário deve ser exigida para utilização do sistema.
Prioridade	Importante
Operação	Entrada
Ator	Usuário

Fonte: Elaborada pelo autor

Quadro 25 – RNF-06 – Separação estrita entre as três camadas arquiteturais

Código	RNF-06
Grupo	Manutenibilidade
Descrição	O sistema deve respeitar a separação estrita entre camada de dados (PostgreSQL), camada de lógica de negócio (NestJS/AC-3) e camada de apresentação (Next.js), de modo que cada camada possa ser modificada de forma independente das demais. Restrições: 1. Alterações na interface não devem requerer modificação do motor de inferência, e vice-versa. 2. A comunicação entre camadas deve ocorrer exclusivamente via API REST.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 26 – RNF-07 – Motor de inferência expansível para novas restrições

Código	RNF-07
Grupo	Manutenibilidade
Descrição	A implementação do AC-3 deve operar sobre restrições definidas como dados, permitindo a adição de novas restrições binárias ao grafo sem refatoração da lógica do motor. Restrições: 1. Novas restrições devem poder ser cadastradas na base de conhecimento sem alteração de código. 2. O grafo de restrições deve ser construído dinamicamente a partir dos dados persistidos no banco. 3. O motor deve aplicar o operador definido em cada restrição de forma genérica, independente da característica comparada.
Prioridade	Importante
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 27 – RNF-08 – Base de conhecimento genérica e reutilizável

Código	RNF-08
Grupo	Manutenibilidade
Descrição	O modelo de dados deve ser genérico, permitindo a inclusão de novas categorias de componentes além das iniciais (CPU, Placa-mãe, RAM, GPU e Fonte) e, em princípio, a reutilização da estrutura em outros domínios de configuração, sem quebra do esquema existente. Restrições: 1. A estrutura da base de conhecimento não deve ser <i>hardcoded</i> para um número fixo de categorias. 2. As categorias e suas características devem ser representadas como dados, não como classes especializadas. 3. O motor de inferência deve operar genericamente sobre qualquer grafo de restrições fornecido.
Prioridade	Desejável
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 28 – RNF-09 – Restrições derivadas de documentação oficial dos fabricantes

Código	RNF-09
Grupo	Confiabilidade
Descrição	Todas as restrições de compatibilidade cadastradas na base de conhecimento devem ser extraídas e referenciadas à documentação técnica oficial de AMD, Intel e JEDEC, garantindo a fidelidade técnica das regras do motor de inferência. Restrições: 1. Cada regra de compatibilidade deve ter sua fonte documental identificável. 2. Regras sem respaldo em documentação oficial de fabricante não devem ser incluídas na base de conhecimento.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 29 – RNF-10 – Determinismo do motor de inferência

Código	RNF-10
Grupo	Confiabilidade
Descrição	Para o mesmo estado de seleções de entrada, o sistema deve sempre produzir o mesmo conjunto de componentes válidos e as mesmas justificativas de bloqueio, sem qualquer variação não determinística. Restrições: 1. O motor de inferência deve ser estritamente determinístico: mesma entrada produz mesma saída. 2. Não deve haver comportamento aleatório, probabilístico ou dependente de estado externo volátil.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 30 – RNF-11 – API *stateless* (sem estado de sessão no servidor)

Código	RNF-11
Grupo	Escalabilidade e Operação
Descrição	A API REST deve operar sem manter estado de sessão entre requisições, recebendo o estado completo das seleções do usuário a cada chamada, em conformidade com o princípio <i>stateless</i> definido por Fielding (2000). Restrições: 1. Nenhum dado de sessão de usuário deve ser armazenado no servidor entre requisições. 2. Cada requisição deve ser autossuficiente e conter todo o contexto necessário para o processamento.
Prioridade	Essencial
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 31 – RNF-12 – Ambiente de banco de dados reproduzível via Docker

Código	RNF-12
Grupo	Escalabilidade e Operação
Descrição	O provisionamento do PostgreSQL deve ser realizado via Docker, garantindo consistência entre os ambientes de desenvolvimento e produção de forma declarativa e reproduzível. Restrições: 1. O ambiente do banco de dados deve ser definido por arquivo de configuração Docker (docker-compose). 2. A instalação direta do PostgreSQL no sistema operacional do desenvolvedor não deve ser necessária.
Prioridade	Importante
Operação	Processamento
Ator	—

Fonte: Elaborada pelo autor

Quadro 32 – RNF-13 – Rastreabilidade de cada bloqueio até a restrição violada

Código	RNF-13
Grupo	Transparência (XAI)
Descrição	O sistema deve ser capaz de identificar e comunicar, para cada componente bloqueado, qual restrição específica do CSP foi violada, alinhando-se ao princípio de transparência da Inteligência Artificial Explicável (Barredo Arrieta <i>et al.</i> , 2020). Restrições: 1. Cada bloqueio deve ser rastreável até a aresta do grafo de restrições que o originou. 2. O sistema deve ser auditável: o raciocínio do motor de inferência deve poder ser inspecionado e verificado.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

Quadro 33 – RNF-14 – Ausência de bloqueios silenciosos

Código	RNF-14
Grupo	Transparência (XAI)
Descrição	Nenhum bloqueio de componente deve ocorrer sem exibição de justificativa ao usuário. O sistema não deve implementar filtragem silenciosa, diferenciando-se explicitamente de plataformas como PC Part Picker e Meupc.net, que sinalizam incompatibilidades sem explicá-las (Clancey, 1983). Restrições: 1. A ausência de justificativa para qualquer bloqueio constitui falha de requisito. 2. A <i>explanation facility</i> deve ser tratada como característica nuclear do sistema, não como funcionalidade acessória.
Prioridade	Essencial
Operação	Saída
Ator	Sistema

Fonte: Elaborada pelo autor

5.2.3 Restrições, Suposições e Dependências

O sistema possui as seguintes dependências e restrições de desenvolvimento:

- As regras de compatibilidade devem ser extraídas exclusivamente da documentação técnica oficial dos fabricantes AMD, Intel e JEDEC;
- O banco de dados PostgreSQL deve estar provisionado via Docker antes da execução do sistema;

- O catálogo inicial de componentes deve ser populado via *Database Seeding* no Prisma ORM antes dos testes de validação.

5.2.4 Requisitos Adiados

Os seguintes requisitos foram identificados, mas serão desenvolvidos em momento posterior:

- Interface administrativa para cadastro e edição de componentes, características e restrições via *front-end* (CRUD completo);
- Ampliação do catálogo para além das cinco categorias iniciais (ex.: cooler, SSD, gabinete);
- Geração de *link* compartilhável com a configuração de *hardware* finalizada;
- Estratégia de testes automatizados (unitários do AC-3 e de integração da API REST).

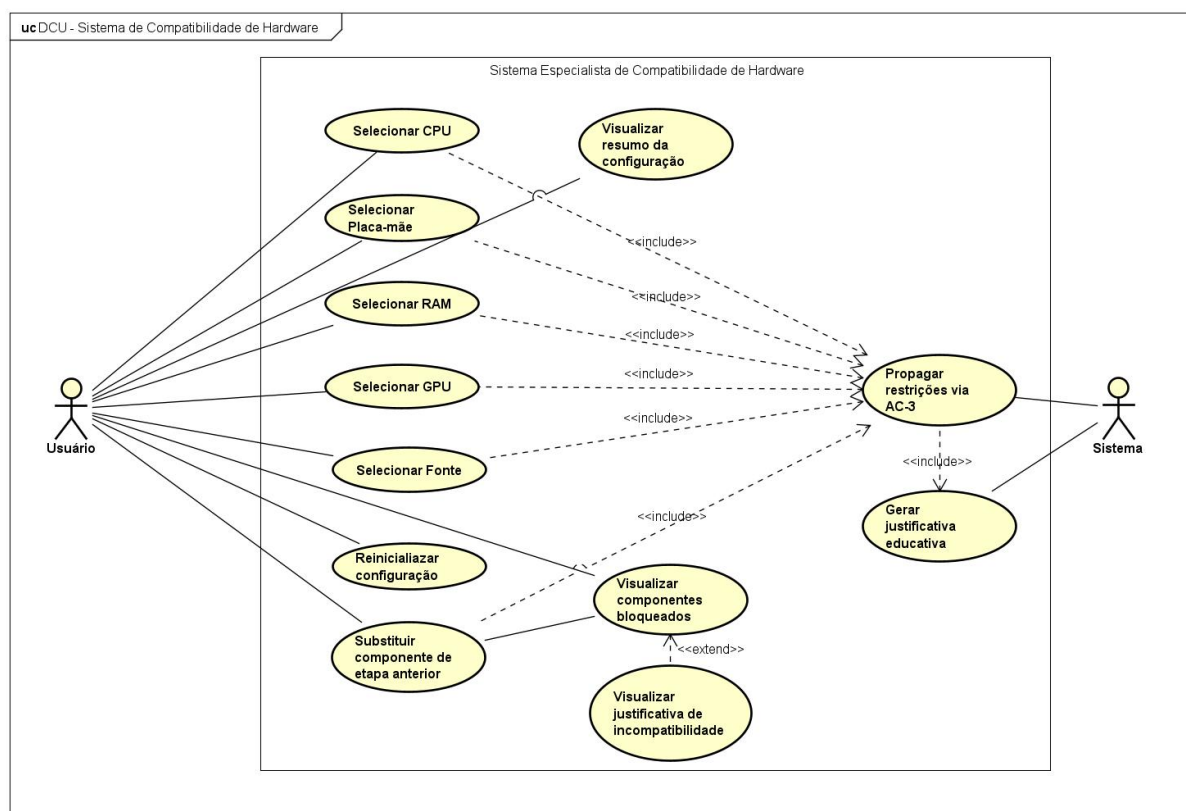
5.3 Casos de Uso

De acordo com Sommerville (2011), os casos de uso são uma técnica de elicitação e descrição de requisitos que identifica os atores envolvidos em uma interação e atribui um nome ao tipo de interação, descrevendo o conjunto de cenários que o sistema deve suportar. Esta seção apresenta os casos de uso identificados neste trabalho, derivados diretamente dos requisitos funcionais descritos anteriormente.

5.3.1 Diagrama de Casos de Uso

O diagrama de casos de uso representa graficamente os atores do sistema, os casos de uso e os relacionamentos entre eles, oferecendo uma visão geral das funcionalidades disponíveis (Sommerville, 2011). O diagrama de casos de uso elaborado neste trabalho está apresentado na Figura 4.

Figura 4 – Diagrama de Casos de Uso



Fonte: Elaborada pelo autor

5.3.2 Especificação dos Casos de Uso

Uma especificação de caso de uso fornece detalhe textual para um caso de uso. A Tabela 1 apresenta o modelo de estrutura de tópicos adotado para a especificação dos casos de uso deste trabalho (extraído de IBM-Especificação de Casos de Uso).

Tabela 1 – Modelo para Especificação de Casos de Uso

Item	Descrição
Nome do Caso de Uso	Declara o nome do caso de uso. Geralmente, o nome expressa o objetivo ou resultado observável do caso de uso, como “Sacar Dinheiro” no caso de um caixa eletrônico.
Descrição Resumida	Descreve a função e o objetivo do caso de uso.
Fluxo de Eventos	Apresenta o fluxo básico e os fluxos alternativos. O fluxo de eventos descreve o comportamento do sistema; ele não descreve como o sistema funciona, os detalhes da apresentação ou os detalhes da interface com o usuário. Se forem trocadas informações, o caso de uso deverá ser específico sobre o que será transmitido de um lado para outro. Por exemplo, em vez de descrever uma ação como “o agente insere informações do cliente”, indique que “o agente insere o nome e o endereço do cliente”.
Fluxo Básico	Descreve o comportamento principal ideal do sistema.
Fluxos Alternativos	Descreve exceções ou desvios do fluxo básico, como o comportamento do sistema quando o agente insere um ID de usuário incorreto e a autenticação do usuário falha.
Requisitos Especiais	Requisitos não-funcionais que são específicos para um caso de uso mas que não são especificados no texto do fluxo do caso de uso de eventos. Exemplos de requisitos especiais incluem estes fatores: requisitos legais e regulamentares; padrões de aplicativo; atributos de qualidade do sistema, incluindo usabilidade, confiabilidade, desempenho e capacidade de suporte; sistemas operacionais e ambientes; requisitos de compatibilidade e restrições de design.
Condições Prévias	Um estado do sistema que deve estar presente antes de um caso de uso ser iniciado.
Pós-Condições	Uma lista de estados possíveis para o sistema imediatamente após a conclusão de um caso de uso.
Pontos de Extensão	Um ponto no fluxo de eventos do caso de uso em que outro caso de uso é referenciado.

Fonte: Extraído de IBM-Especificação de Casos de Uso

A título de exemplo, a Tabela 2 apresenta a especificação do caso de uso *Selecionar Componente*, central ao fluxo de configuração do sistema.

Tabela 2 – Especificação do Caso de Uso “Selecionar Componente”

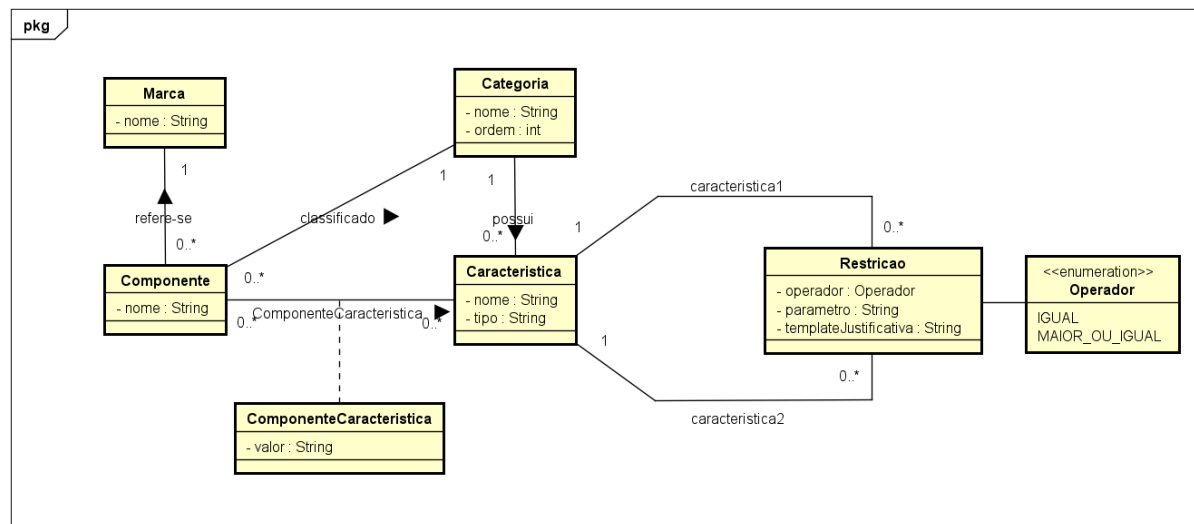
Caso de Uso Selecionar Componente
Referências RF-01, RF-05, RF-09, RF-10
Descrição Geral O caso de uso inicia-se quando o usuário seleciona um componente em uma das etapas do <i>wizard</i> de configuração. O sistema dispara a propagação de restrições e atualiza a disponibilidade dos componentes nas etapas seguintes.
Atores Usuário, Sistema
Pré-Condições A base de conhecimento deve estar populada; o usuário deve estar em uma etapa válida do <i>wizard</i> .
Garantia de Sucesso (Pós-Condições) Componente registrado na configuração; domínios das categorias seguintes atualizados; componentes incompatíveis sinalizados com a respectiva justificativa.
Requisitos Especiais A propagação deve ocorrer em tempo real (RNF-01); nenhum bloqueio pode ocorrer sem justificativa (RNF-14).
Fluxo Básico 1. Usuário visualiza a lista de componentes da categoria atual; 2. Usuário seleciona um componente compatível; 3. Sistema registra a seleção e envia o estado completo à API REST; 4. Sistema executa o algoritmo AC-3 sobre o grafo de restrições; 5. Sistema poda os domínios das categorias seguintes e retorna os componentes disponíveis; 6. Sistema apresenta a etapa seguinte com os componentes atualizados.
Fluxo Alternativo 1. Usuário tenta selecionar um componente incompatível 1.1 Sistema bloqueia a seleção e exibe a justificativa técnica da incompatibilidade (RF-10); 1.2 Retorna ao passo 1.

5.4 Modelo de Domínio

O modelo de domínio é um artefato de análise que representa os conceitos relevantes do domínio do problema e os relacionamentos entre eles, sem preocupação com detalhes de implementação (Larman, 2004). Por se concentrar no vocabulário do domínio, o modelo é elaborado apenas com as classes e seus atributos, auxiliando na compreensão da aplicação. A Figura 5 apresenta o modelo de domínio desenvolvido neste trabalho.

O modelo reflete a decisão arquitetural de tornar a tripla $\langle X, D, C \rangle$ do CSP configurável. Em vez de especializar a classe *Componente* em subclasses fixas (CPU, Placa-mãe, RAM, GPU e Fonte), com atributos codificados, adota-se uma estrutura genérica baseada em metadados: a classe *Categoria* representa as variáveis (X); a classe *Componente* representa os valores de domínio (D); a classe *Caracteristica* define, por categoria, os atributos configuráveis, cujos valores são registrados em *ComponenteCaracteristica*; e a classe *Restricao* representa o conjunto C , associando duas características por meio de um operador. Essa organização realiza concretamente os requisitos RNF-07 e RNF-08, uma vez que a inclusão de novas categorias, características ou restrições não exige alteração de código, apenas a inserção de novos registros na base de conhecimento.

Figura 5 – Modelo de Domínio



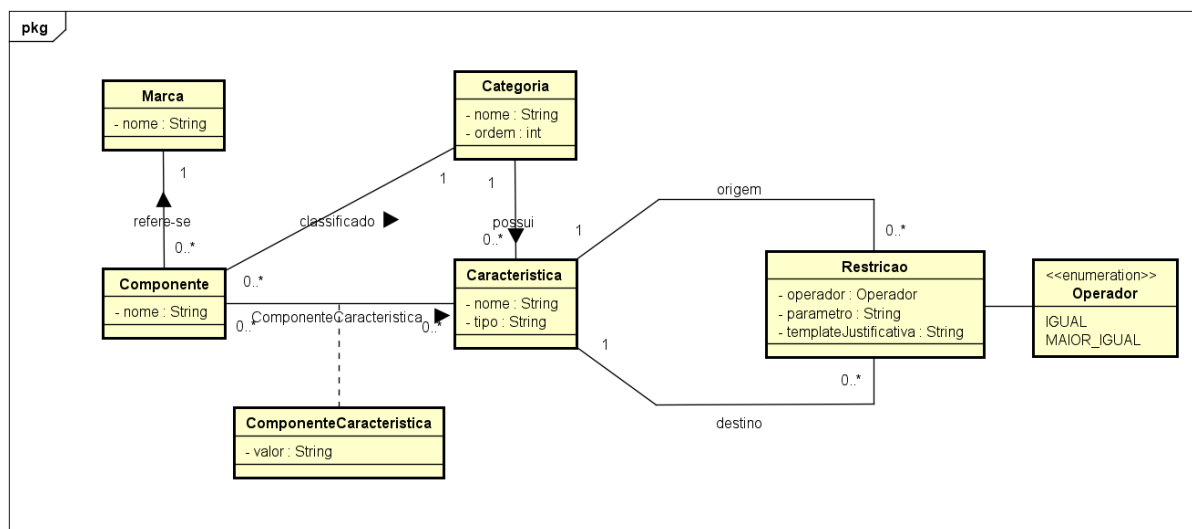
Fonte: Elaborada pelo autor

5.5 Diagrama de Classes de Análise

O diagrama de classes de análise é uma evolução do modelo de domínio que incorpora, além dos atributos, os métodos que descrevem o comportamento das classes, aproximando o modelo da estrutura que será implementada (Larman, 2004). A Figura 6 apresenta o diagrama de classes de análise elaborado neste trabalho.

Em relação ao modelo de domínio, o diagrama acrescenta os métodos responsáveis pela lógica do sistema. A classe *Restricao* recebe o método *isSatisfeita*, que avalia se os valores de duas características atendem ao operador definido; a classe *ComponenteCaracteristica* expõe o método de acesso ao valor; e são introduzidas as classes de saída do motor de inferência, *ResultadoValidacao* e *JustificativaEducativa*, responsáveis por transportar, respectivamente, o resultado da propagação e as justificativas educativas exibidas ao usuário. Diferentemente das classes da base de conhecimento, essas duas classes não são persistidas, sendo geradas a cada execução do AC-3.

Figura 6 – Diagrama de Classes de Análise



Fonte: Elaborada pelo autor

5.6 Diagrama de Atividades

Um diagrama de atividades ilustra a natureza dinâmica de um sistema pela modelagem do fluxo de controle de uma atividade à outra. Uma atividade representa uma operação em alguma classe do sistema que resulta em uma mudança de estado. Tipicamente, diagramas de atividades são usados para modelar fluxos de processos, processos de negócios ou operações internas. Embora seja semelhante a uma máquina de estados, o diagrama de atividades tem propósito distinto, voltado a capturar ações e seus resultados em termos de mudanças no estado do objeto.

O fluxo é mostrado por meio de transições, representadas por setas direcionadas que indicam o caminho entre os estados de atividade (ação). Um diagrama de atividades é normalmente composto pelos seguintes elementos: atividades (ações), estados de atividade (ação), transição, fluxo de objeto, estado inicial, estado final, *branching*, sincronização e raias.

6 PROJETO DE SOFTWARE

Este capítulo apresenta as decisões de projeto que traduzem os requisitos e os modelos de análise em uma representação concreta do sistema, abrangendo o projeto de interface, o projeto de dados, o projeto procedimental e o projeto arquitetural.

6.1 Projeto de Interface

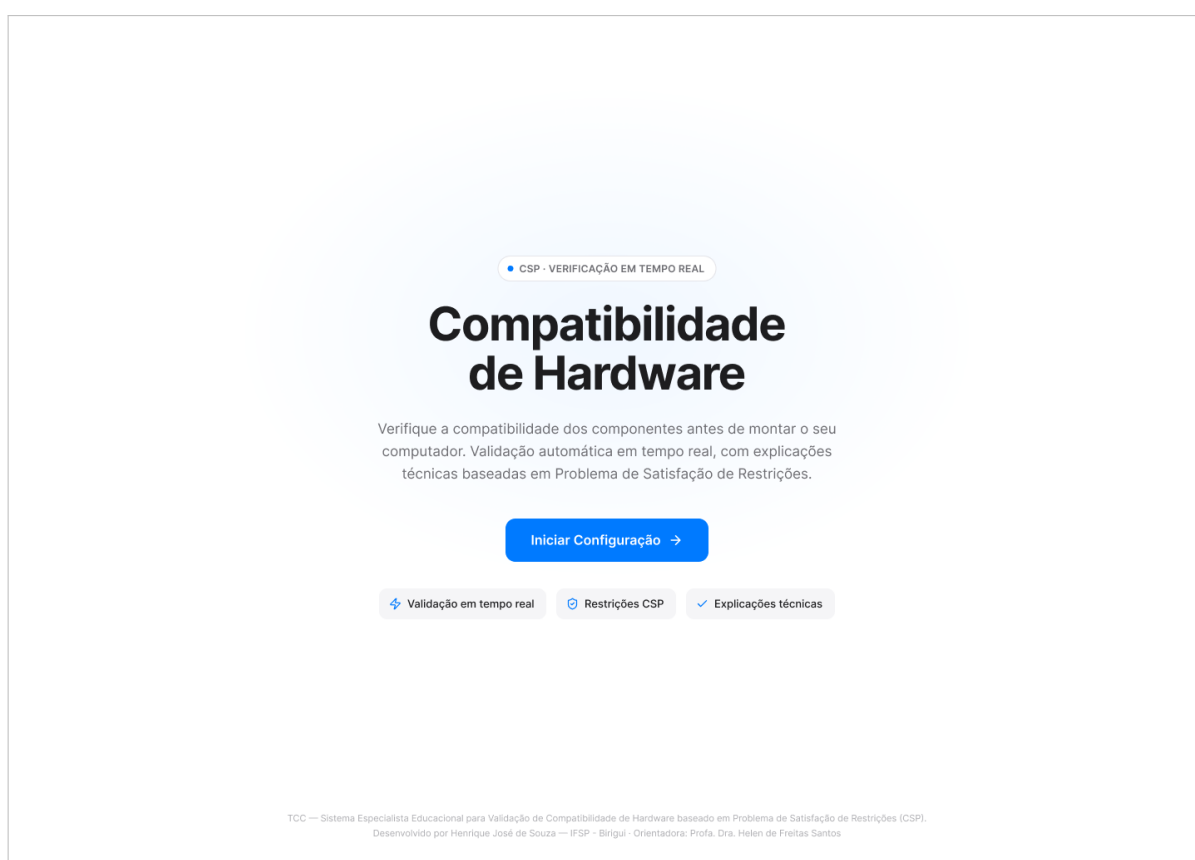
A camada de apresentação foi projetada como uma interface no formato *wizard*, que conduz o usuário sequencialmente pela seleção das cinco categorias de componentes (CPU, Placa-mãe, RAM, GPU e Fonte). Os protótipos foram elaborados na ferramenta Figma antes da implementação, o que permitiu validar o *layout* e o fluxo de navegação ainda na fase de projeto.

A construção visual partiu de um *layout* estruturado sobre um grid de oito colunas na resolução base de 1440×1024, com margens laterais de 128px e espaçamento interno de 32px entre colunas. Sobre essa base, adotou-se a biblioteca *shadcn/ui* apoiada no Tailwind CSS, com *design tokens* centralizados para cores, tipografia e raios de borda, de modo a manter a consistência visual entre todas as telas.

Uma decisão central de interface diz respeito ao tratamento dos componentes incompatíveis. Em vez de ocultá-los da listagem, o sistema os mantém visíveis acompanhados de um indicador de bloqueio, revelando a justificativa técnica no momento em que o usuário interage com eles. Essa escolha é coerente com o caráter educativo do trabalho, pois transforma cada bloqueio em uma oportunidade de aprendizado em vez de uma simples ausência.

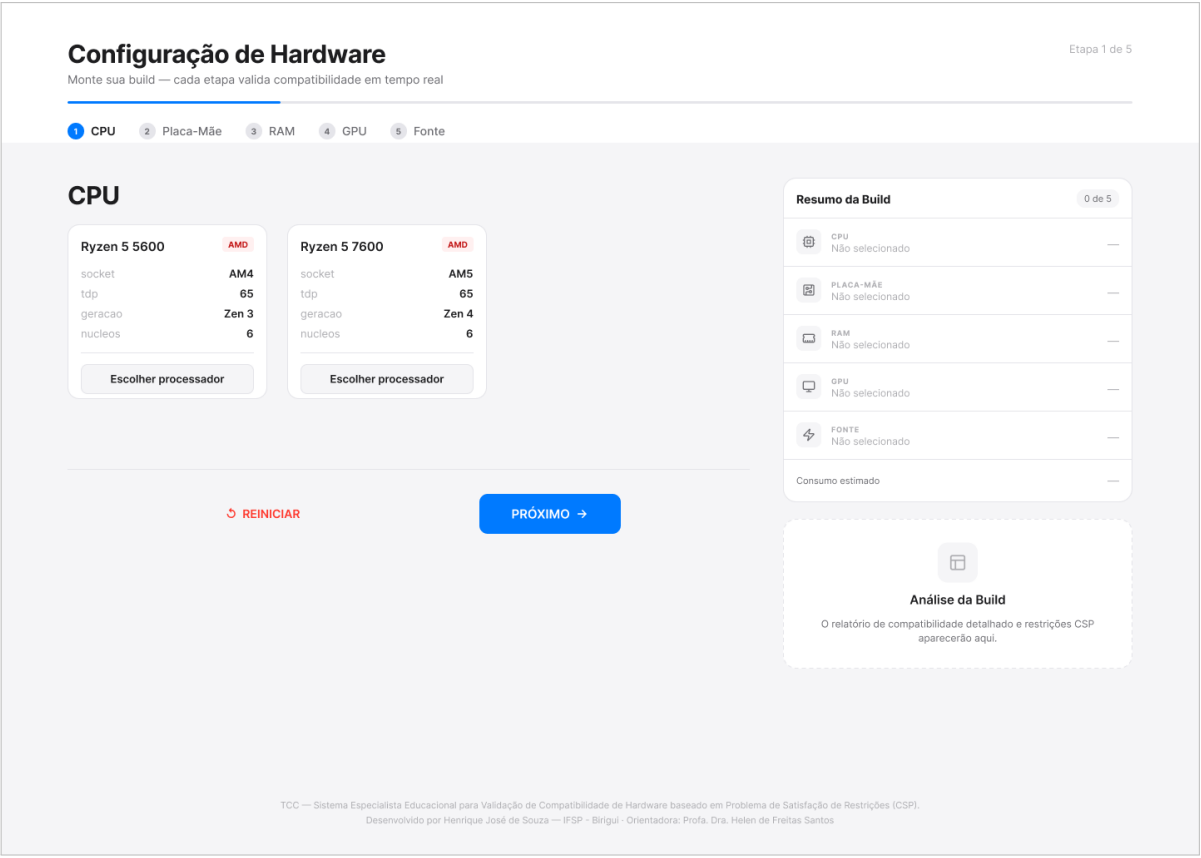
As Figuras 7 a 9 apresentam os protótipos das três principais telas do sistema em sua versão para *desktop*. A primeira é a tela inicial, que recebe o usuário e dá início ao fluxo de configuração. A segunda é a tela de seleção de componentes, na qual o usuário escolhe cada categoria de *hardware* e visualiza, em tempo real, a sinalização das incompatibilidades detectadas. A terceira é a tela de resultado, que consolida a configuração montada e apresenta o seu status geral de compatibilidade.

Figura 7 – Tela inicial do sistema



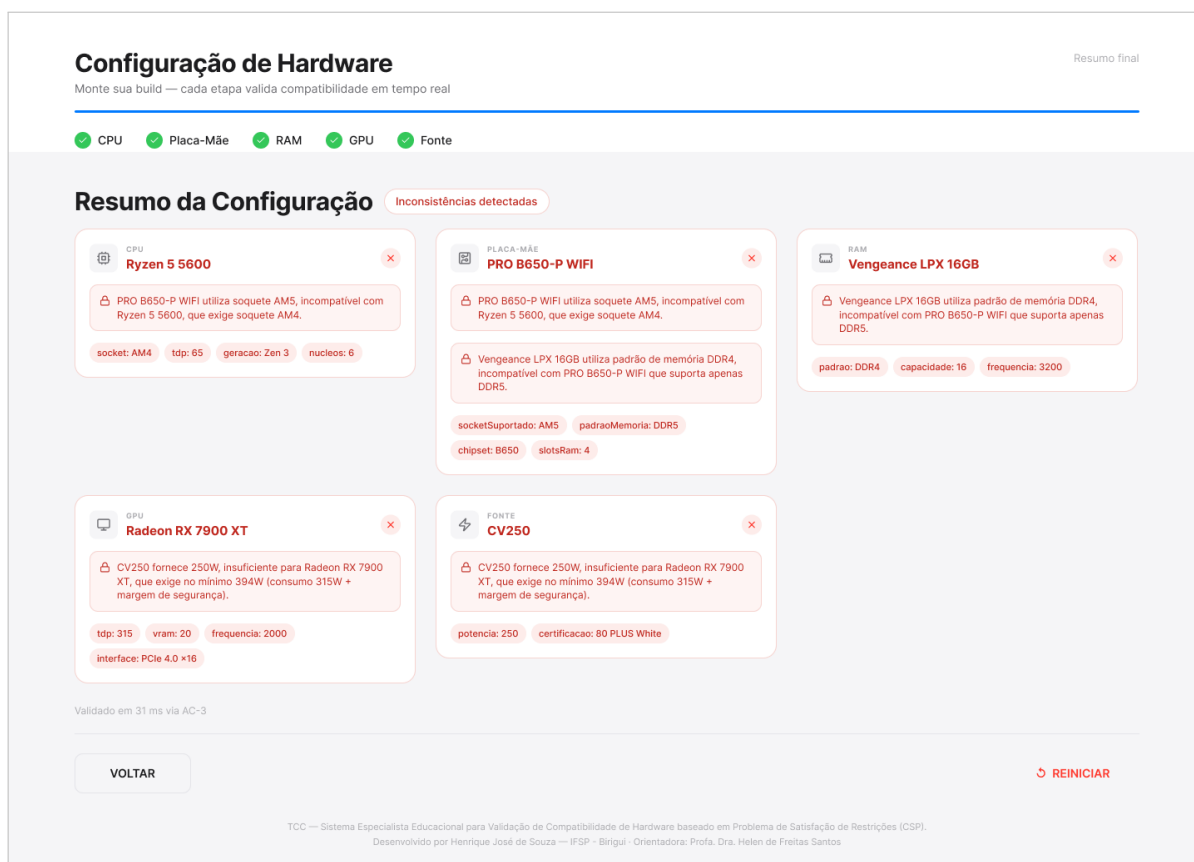
Fonte: Elaborada pelo autor

Figura 8 – Tela de seleção de componentes com sinalização de incompatibilidade



Fonte: Elaborada pelo autor

Figura 9 – Tela de resumo da configuração finalizada



Fonte: Elaborada pelo autor

Em conformidade com o requisito de responsividade, a interface foi desenvolvida para adaptar-se a diferentes resoluções de tela, preservando todas as funcionalidades em dispositivos móveis. As Figuras 10 a 12 apresentam as mesmas três telas em sua versão para dispositivo móvel, na qual os elementos são reorganizados verticalmente para acomodar a largura reduzida sem prejuízo da navegação pelo *wizard*.

Figura 10 – Tela inicial em dispositivo móvel



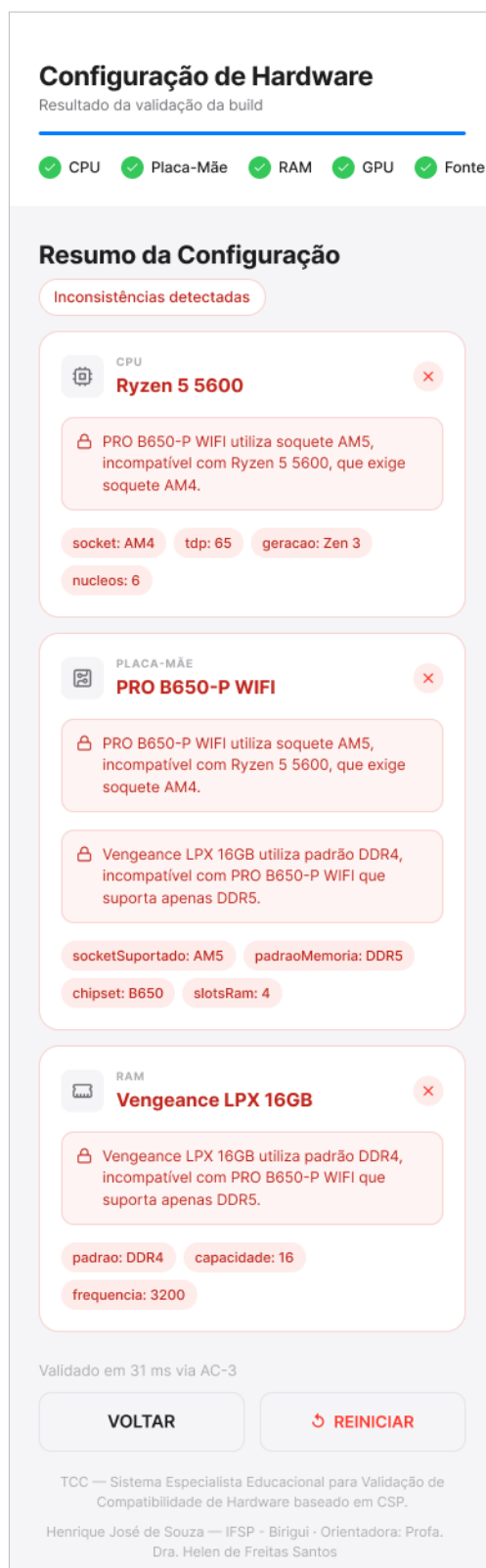
Fonte: Elaborada pelo autor

Figura 11 – Tela de seleção de componentes em dispositivo móvel



Fonte: Elaborada pelo autor

Figura 12 – Tela de resumo da configuração em dispositivo móvel



Fonte: Elaborada pelo autor

6.2 Projeto de Dados

O sistema utiliza o banco de dados relacional PostgreSQL, versão 16, executado em *container* Docker, o que garante um ambiente reproduzível e consistente entre desenvolvimento e produção. Durante o desenvolvimento, a manipulação e a inspeção dos dados foram feitas com a ferramenta DBeaver, em sua *Community Edition*. O acesso ao banco, a partir da camada de lógica de negócio, é intermediado pelo Prisma ORM, responsável por mapear os objetos da aplicação para as tabelas relacionais.

6.2.1 Mapeamento Objeto-Relacional

Uma vez que a modelagem foi elaborada de forma orientada a objetos, por meio do diagrama de classes UML, e o banco de dados adotado é relacional, foi necessário identificar as relações que compõem o esquema. O mapeamento partiu do modelo de domínio e seguiu um conjunto de regras simples. Cada classe concreta deu origem a uma tabela, e cada atributo simples tornou-se uma coluna com o tipo de dado relacional equivalente, enquanto o identificador único de cada classe foi adotado como chave primária. As associações do tipo um-para-muitos foram representadas por uma chave estrangeira na tabela do lado de multiplicidade muitos, e as associações muitos-para-muitos deram origem a tabelas associativas que reúnem as chaves estrangeiras das duas entidades envolvidas.

A decisão de projeto mais relevante nesse mapeamento foi a forma de representar as especificações técnicas, que variam de uma categoria de componente para outra. Em vez de criar uma tabela específica para cada categoria, com colunas fixas, optou-se pelo padrão Entidade-Atributo-Valor (EAV), no qual as características técnicas são armazenadas como dados e não como estrutura. Essa abordagem permite incluir novas categorias e novas características apenas inserindo registros, sem qualquer alteração no esquema do banco, o que sustenta diretamente a expansibilidade pretendida para a base de conhecimento.

A partir dessas regras, foram identificadas as relações apresentadas na Tabela 4.

6.2.2 Estrutura das Tabelas no Banco de Dados

Para padronizar a nomenclatura dos objetos do banco de dados, foi adotada a convenção definida na Tabela 3, que cobre as chaves primárias e estrangeiras, as *constraints* de verificação e as chaves únicas.

Tabela 3 – Convenção para Nome dos Objetos no Banco de Dados

Objeto	Padrão Adotado
Chave Primária	NomeDaTabela_PK
Chave Estrangeira	NomeDaTabela_NomeDaTabelaEstrangeira_FK_nn, onde nn é a sequência de ocorrência do par NomeDaTabela e NomeDaTabelaEstrangeira
Check	NomeDaTabela_CK_nn, onde nn é a sequência de checks da tabela
Chave Única	NomeDaTabela_UK_nn, onde nn é a sequência de chave única da tabela

Fonte: Elaborada pelo autor

As tabelas que compõem a base de conhecimento do sistema especialista estão consolidadas na Tabela 4 e detalhadas individualmente nas tabelas seguintes.

Tabela 4 – Tabelas Identificadas neste Trabalho

Tabela do Banco de Dados	Tabela no Documento
Marca	Tabela 6
Categoria	Tabela 7
Caracteristica	Tabela 8
Componente	Tabela 9
ComponenteCaracteristica	Tabela 10
Restricao	Tabela 11

Fonte: Elaborada pelo autor

A modelagem emprega dois tipos enumerados, descritos na Tabela 5, utilizados como domínio fechado para os campos **tipo** (em *Caracteristica*) e **operador** (em *Restricao*).

Tabela 5 – Tipos Enumerados do Modelo de Dados

Enumerado	Valor	Significado
TipoCaracteristica	TEXTO	Valor comparado como texto (ex.: <i>socket</i> , padrão DDR).
	INTEIRO	Valor convertido para número antes da comparação (ex.: <i>tdp</i> , potência).
OperadorRestricao	IGUAL	Satisfeita quando os dois valores são idênticos (restrição binária absoluta).
	MAIOR_OU_IGUAL	Satisfeita quando capacidade $\geq$ demanda $\times$ parâmetro (restrição quantitativa).

Fonte: Elaborada pelo autor

Tabela 6 – Marca

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
nome	Varchar	X				UK_01	1

Fonte: Elaborada pelo autor

Tabela 7 – Categoria

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
nome	Varchar	X				UK_01	1
ordem	Integer	X					

Fonte: Elaborada pelo autor

Tabela 8 – Caracteristica

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
idCategoria	UUID	X		Categoria	id	UK_01	1
nome	Varchar	X				UK_01	2
tipo	TipoCaracteristica	X					

Fonte: Elaborada pelo autor

Tabela 9 – Componente

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
idCategoria	UUID	X		Categoria	id		
idMarca	UUID	X		Marca	id		
nome	Varchar	X					

Fonte: Elaborada pelo autor

Tabela 10 – ComponenteCaracteristica

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
idComponente	UUID	X		Componente	id	UK_01	1
idCaracteristica	UUID	X		Caracteristica	id	UK_01	2
valor	Varchar	X					

Fonte: Elaborada pelo autor

Tabela 11 – Restricao

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo	Ordem
id	UUID	X	X				
idCaracteristica1	UUID	X		Caracteristica	id		
idCaracteristica2	UUID	X		Caracteristica	id		
operador	OperadorRestricao	X					
parametro	Varchar						
templateJustificativa	Text	X					

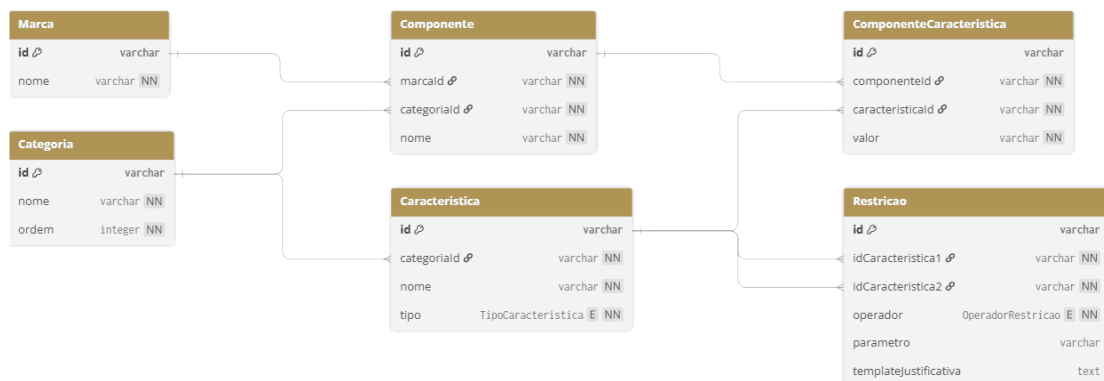
Fonte: Elaborada pelo autor

A tabela *Restricao* é o elemento central desse modelo, pois cada um de seus registros corresponde a uma aresta do grafo de restrições, ou seja, ao conjunto C do CSP.

Os campos `idCaracteristica1` e `idCaracteristica2` referenciam, respectivamente, a característica do lado da demanda e a do lado da capacidade, o que permite ao motor AC-3 montar o grafo de forma dinâmica a partir dos próprios dados, sem precisar de regras condicionais escritas no código para cada tipo de restrição. O campo `parametro` é opcional e aplica-se apenas às restrições quantitativas do tipo `MAIOR_OU_IGUAL`, guardando o fator multiplicador da margem de segurança, como o valor "1.25" usado para os 25% recomendados no dimensionamento da fonte. Já o campo `templateJustificativa` guarda o texto da explicação educativa em linguagem natural, com marcadores que são substituídos em tempo de execução pelos valores dos componentes envolvidos. Dessa forma, nenhum termo técnico do domínio fica embutido no código, e toda a explicação nasce do próprio dado cadastrado.

Para complementar a descrição textual das tabelas, a Figura 13 apresenta o diagrama entidade-relacionamento da base de conhecimento, gerado a partir do esquema físico efetivamente implementado no PostgreSQL. O diagrama evidencia as seis relações do modelo e as chaves estrangeiras que as conectam, tornando visível a estrutura genérica baseada no padrão Entidade-Atributo-Valor adotada no trabalho.

Figura 13 – Diagrama Entidade-Relacionamento da base de conhecimento



Fonte: Elaborada pelo autor

A leitura do diagrama revela o papel central da entidade **Categoria**, que se relaciona tanto com **Componente** quanto com **Caracteristica**, refletindo sua função de representar as variáveis do CSP. A entidade **ComponenteCaracteristica** materializa a associação entre um componente e o valor de cada uma de suas características, concretizando o modelo Entidade-Atributo-Valor que permite atribuir características distintas a categorias distintas sem alteração do esquema. A entidade **Restricao**, por sua vez, conecta-se duas vezes à entidade **Caracteristica**, uma como característica de origem e outra como característica de destino, o que expressa cada aresta do grafo de restrições diretamente na estrutura do banco de dados.

6.2.3 Diagrama de Pacotes

A organização do código em pacotes reflete a separação arquitetural em camadas, com módulos distintos para a apresentação, a lógica de negócio e o acesso a dados.

6.2.4 Diagrama de Classes de Projeto

O diagrama de classes de projeto detalha as classes da implementação, incluindo atributos, métodos e tipos concretos, evoluindo o diagrama de classes de análise apresentado no Capítulo 5.

6.3 Projeto Procedimental

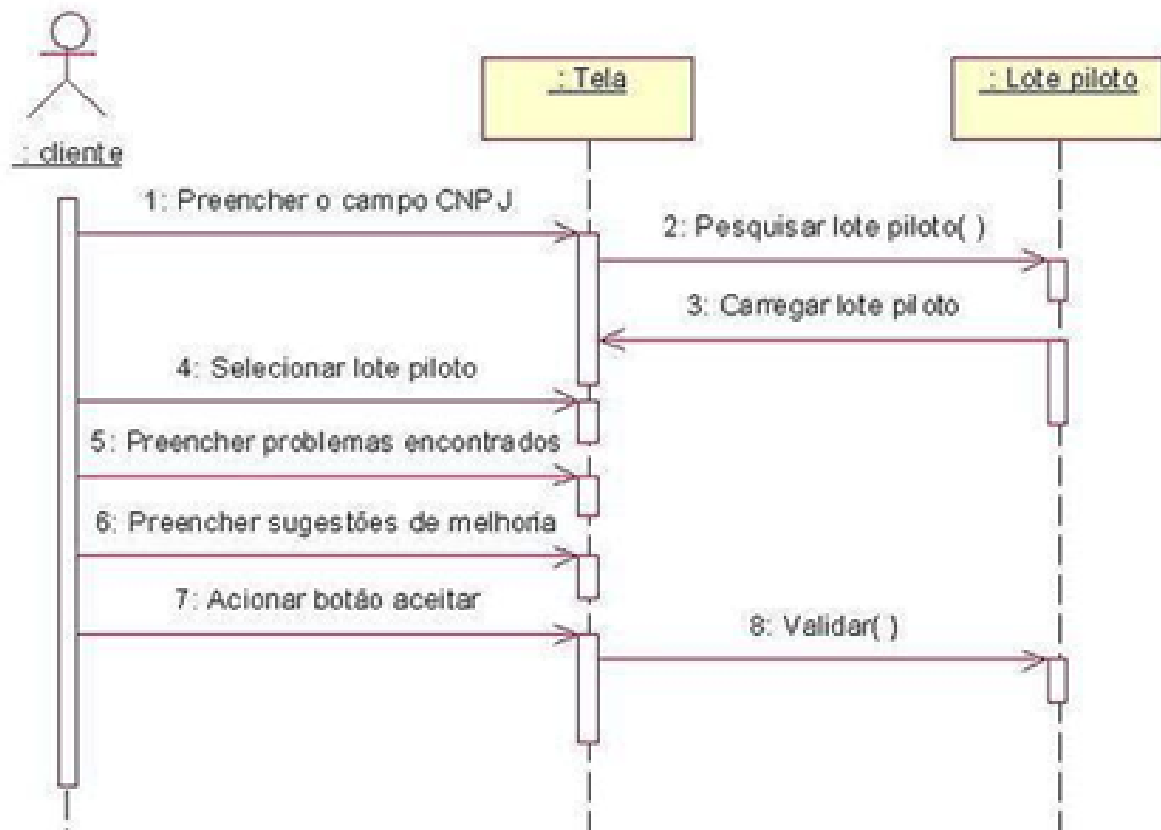
O projeto procedimental detalha o comportamento interno do sistema, traduzindo os requisitos e os modelos de análise em representações que descrevem o fluxo de controle e a interação entre as partes do software (Pressman; Maxim, 2016). Nesta seção, esse detalhamento é apresentado por meio do diagrama de sequência, que descreve a interação entre as camadas durante uma requisição de validação.

6.3.1 Diagrama de Sequência

Os diagramas de sequência modelam as interações entre objetos em um caso de uso, ilustrando como as diferentes partes do sistema interagem para realizar uma função e a ordem em que essas interações ocorrem (**creately**). A Figura 14 apresenta o diagrama de sequência para o caso de uso *Propagar restrições via AC-3*.

Ao selecionar um componente, a camada de apresentação (Next.js) envia o estado completo das seleções à API REST (NestJS). A camada de lógica de negócio executa o algoritmo AC-3 sobre o grafo de restrições, consultando a base de conhecimento (PostgreSQL) para obter os domínios e as restrições aplicáveis. Em seguida, poda os valores inconsistentes e, para cada bloqueio detectado, aciona o módulo de geração de justificativas. O resultado — domínios atualizados e justificativas técnicas — é retornado à camada de apresentação, que atualiza a exibição em tempo real.

Figura 14 – Diagrama de Sequência



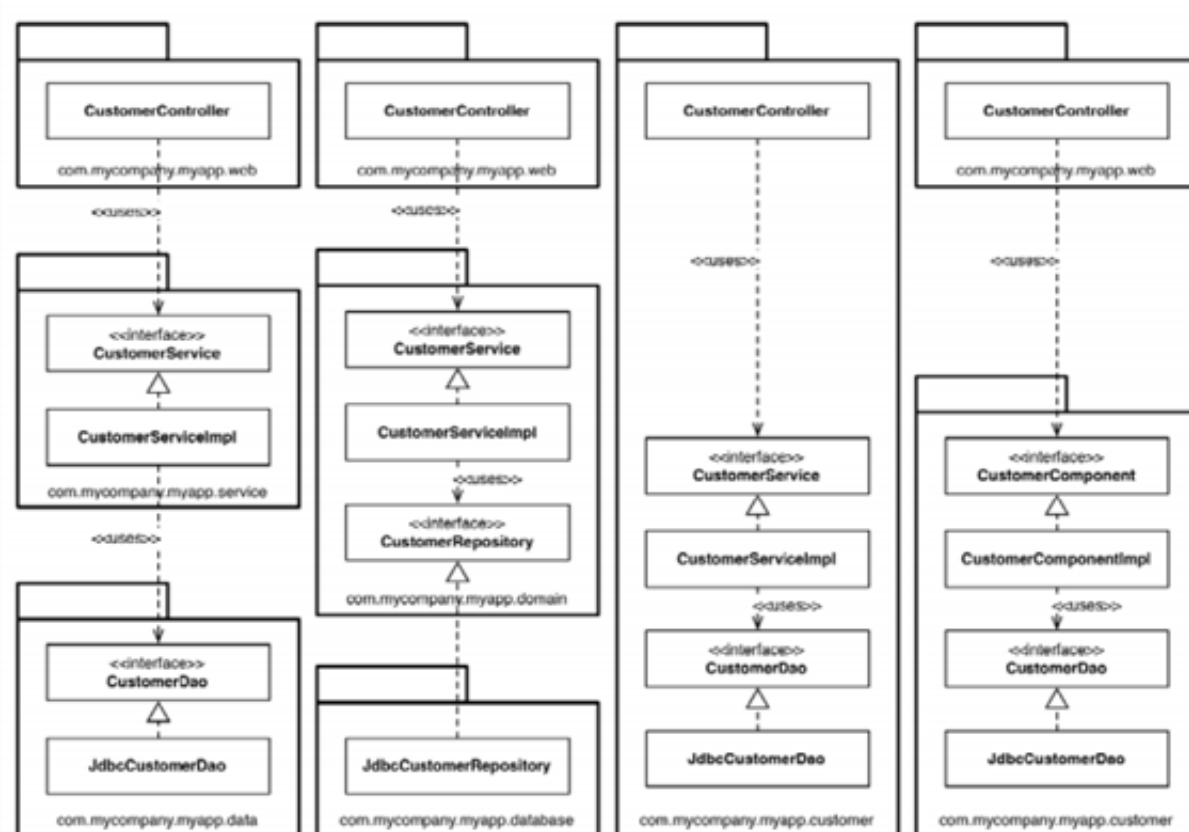
Fonte: Elaborada pelo autor

6.4 Projeto Arquitetural

O sistema adota o padrão arquitetural de três camadas (Fowler, 2002), mantendo uma separação clara entre apresentação, lógica de negócio e dados. A camada de apresentação, construída em Next.js, é responsável pela interface *wizard* com a qual o usuário interage diretamente. A camada de lógica de negócio, implementada em NestJS, abriga o motor de inferência AC-3 e o módulo de justificativas educativas, sendo o núcleo de processamento do sistema. Por fim, a camada de dados, em PostgreSQL provisionado via Docker, armazena toda a base de conhecimento do sistema especialista.

A comunicação entre as camadas é deliberadamente delimitada. A interface conversa com a lógica de negócio exclusivamente por meio de uma API REST (Fielding, 2000), enquanto a lógica de negócio acessa os dados por intermédio do Prisma ORM. Essa separação garante que o motor de inferência possa ser mantido e expandido de forma independente da interface, preservando a modularidade ao longo de todo o ciclo de desenvolvimento. A organização e a comunicação entre as camadas estão ilustradas na Figura 15.

Figura 15 – Arquitetura em Três Camadas do Sistema



Fonte: Elaborada pelo autor

7 IMPLEMENTAÇÃO

Este capítulo descreve a concretização do sistema especialista educacional cujos requisitos e projeto foram apresentados nos Capítulos 5 e 6. Mais do que relatar quais tecnologias foram empregadas, busca-se justificar as decisões de implementação que sustentam as propriedades centrais do trabalho: a generalidade do motor de inferência, seu determinismo e sua capacidade de explicação. São detalhados o ambiente e as ferramentas de desenvolvimento, a organização do repositório e a implementação de cada uma das três camadas arquiteturais, bem como a interface de comunicação entre elas. Dá-se ênfase à camada de lógica de negócio, que abriga o motor de inferência AC-3 e a *explanation facility*, por constituírem o núcleo da contribuição deste trabalho. Ao final, registram-se as limitações conhecidas da implementação em seu estado atual.

7.1 Ambiente e Ferramentas de Desenvolvimento

A escolha das tecnologias foi orientada por um princípio unificador: adotar um único ecossistema de linguagem em todas as camadas, de modo a permitir o compartilhamento de definições de tipos entre o servidor e o cliente. Por essa razão, o sistema foi desenvolvido integralmente em *TypeScript* (Microsoft Corporation, 2025a), cuja tipagem estática permite que os contratos de dados trafegados entre as camadas sejam verificados em tempo de compilação, reduzindo a classe de erros que só se manifestaria em execução. O ambiente de execução do lado do servidor é o *Node.js* (OpenJS Foundation, 2025), escolhido justamente por executar TypeScript no servidor e, assim, viabilizar esse ecossistema unificado.

A camada de dados é provida pelo *PostgreSQL* (PostgreSQL Global Development Group, 2025) em sua versão 16, selecionado por sua robustez, conformidade com o padrão SQL e adequação ao modelo relacional exigido pela base de conhecimento. Sua execução em contêiner por meio do *Docker* (Docker, Inc., 2025) foi uma decisão deliberada para garantir um ambiente de banco de dados reproduzível e idêntico entre as máquinas de desenvolvimento, atendendo diretamente ao requisito RNF-12 e dispensando a instalação manual do banco no sistema operacional. Durante o desenvolvimento, a inspeção e a manipulação manual dos dados foram realizadas com o *DBeaver* (DBeaver Corp., 2025), e a codificação foi conduzida no *Visual Studio Code* (Microsoft Corporation, 2025b). O Quadro 34 consolida as versões das principais tecnologias empregadas.

Quadro 34 – Versões das principais tecnologias utilizadas

Tecnologia	Versão
Node.js	$\geq 20.0.0$
TypeScript	5.6.3
NestJS	10.4.15
Next.js	15.1.3
React	19.0.0
Prisma ORM	5.22.0
PostgreSQL	16
pnpm	9.15.0
Turborepo	2.3.3

Fonte: Elaborada pelo autor

7.2 Organização do Repositório

O código-fonte foi organizado como um *monorepo*, isto é, um único repositório que abriga múltiplos projetos relacionados, gerenciado pelo *pnpm* (Pnpm, 2025) como gerenciador de pacotes e pelo *Turborepo* (Vercel Inc., 2025b) como orquestrador das tarefas de *build* e execução. A opção pelo monorepo, em vez de repositórios separados para servidor e cliente, decorre diretamente da estratégia de ecossistema unificado: mantendo ambos no mesmo repositório, torna-se possível extrair as definições de tipos compartilhadas para um pacote interno, consumido tanto pelo servidor quanto pelo cliente. Assim, o contrato de comunicação entre as camadas existe em um único lugar, e qualquer alteração é verificada em tempo de compilação em ambos os lados, eliminando a duplicação e a divergência silenciosa de tipos que ocorreria com repositórios independentes.

O repositório organiza-se em dois diretórios principais. O diretório **apps** contém as aplicações executáveis: **api**, que implementa as camadas de lógica de negócio e de dados em NestJS (NestJS, 2025), e **web**, que implementa a camada de apresentação em Next.js (Vercel Inc., 2025a). O diretório **packages** contém os módulos compartilhados, com destaque para **shared-types**, que centraliza as interfaces trafegadas entre cliente e servidor, como **EstadoConfiguracao**, **RespostaValidacao** e **JustificativaEducativa**. A estrutura simplificada do repositório é apresentada no Código 7.1.

Código 7.1 – Estrutura simplificada do monorepo

```

1 apps/
2   api/
3     prisma/          (schema.prisma, migrations/, seed.ts)
4     src/
5       app.module.ts, main.ts
6       categorias/    components/    configurations/
7       csp/           explanations/   prisma/
8   web/

```

```

9      src/
10     app/           (layout.tsx, page.tsx, wizard/page.tsx)
11     components/    (ui/)
12     lib/           (api.ts, utils.ts)
13 packages/
14   shared-types/    (src/index.ts)
15   tsconfig/        (base.json, nestjs.json, nextjs.json)

```

Fonte: Elaborado pelo autor

Essa organização não é meramente estética: ela materializa fisicamente a separação estrita entre as três camadas exigida pelo requisito RNF-06. A camada de dados reside nos módulos `prisma` e `csp` do lado do servidor, a lógica de negócio nos módulos `csp`, `configurations` e `explanations`, e a apresentação integralmente no aplicativo `web`. O pacote `shared-types` não contém lógica de execução, apenas contratos, funcionando como uma fronteira tipada explícita entre as camadas distribuídas de modo que a comunicação entre elas é obrigada a passar por um contrato conhecido, e não por acoplamento direto.

7.3 Camada de Dados

A persistência foi implementada sobre o PostgreSQL, acessado a partir da camada de lógica de negócio por intermédio do *Prisma ORM* (Prisma Data, Inc., 2025), responsável por mapear os registros relacionais em objetos da aplicação. A adoção de um ORM justifica-se por manter a camada de lógica de negócio trabalhando com objetos tipados, sem consultas SQL escritas manualmente, o que preserva a coerência com o ecossistema TypeScript e reduz a superfície de erro no acesso a dados. O esquema do banco reproduz fielmente o modelo genérico baseado no padrão Entidade-Atributo-Valor (EAV) apresentado na Seção 6.2 do Capítulo 6, materializando as seis relações `Marca`, `Categoria`, `Caracteristica`, `Componente`, `ComponenteCaracteristica` e `Restricao`, além dos dois tipos enumerados `TipoCaracteristica` (`TEXTO`, `INTEIRO`) e `OperadorRestricao` (`IGUAL`, `MAIOR_OU_IGUAL`).

A decisão de projeto mais consequente desta camada é a representação das restrições como dados, e não como código. O Código 7.2 apresenta a definição da entidade `Restricao` no *schema* do Prisma, elemento central do modelo por representar cada aresta do grafo de restrições, ou seja, cada elemento do conjunto C do CSP.

Código 7.2 – Definição da entidade `Restricao` no schema do Prisma

```

1 model Restricao {
2   id                String          @id @default(uuid())
3   idCaracteristica1 String
4   idCaracteristica2 String

```

```

5      operador          OperadorRestricao
6      parametro         String?
7      templateJustificativa String
8
9      caracteristica1 Caracteristica @relation("origem", ...)
10     caracteristica2 Caracteristica @relation("destino", ...)
11 }

```

Fonte: Elaborado pelo autor

Ao guardar em cada registro as duas características relacionadas, o operador e o texto da justificativa, uma restrição passa a ser inteiramente descrita por uma linha da tabela. Essa escolha é o que sustenta a expansibilidade exigida pelos requisitos RNF-07 e RNF-08: incluir uma nova restrição, ou mesmo uma nova categoria de componente, reduz-se a inserir registros, sem qualquer alteração no motor de inferência. É essa decisão que concretiza, no nível da persistência, a ambição de um motor genérico e reutilizável.

A gestão do esquema é feita por meio das *migrations* do Prisma, aplicadas com o comando `prisma migrate dev`, o que mantém a evolução do banco versionada e reproduzível. Duas *migrations* compõem o histórico do projeto: a criação inicial do esquema e a subsequente adoção do modelo EAV genérico, transição que reflete, no próprio versionamento, a decisão de abandonar tabelas específicas por categoria em favor da estrutura genérica. A carga inicial da base de conhecimento é realizada por um procedimento de *Database Seeding* (`seed.ts`), em conformidade com o requisito RF-18. O Quadro 35 apresenta a contagem de registros inseridos pela carga inicial, dimensionada para cobrir as plataformas AMD AM4 e AM5 e, com isso, viabilizar os cenários de validação previstos.

Quadro 35 – Registros criados pela carga inicial da base de conhecimento

Entidade	Registros
Marca	5
Categoria	5
Caracteristica	19
Componente	11
ComponenteCaracteristica	40
Restricao	4

Fonte: Elaborada pelo autor

As quatro restrições cadastradas correspondem exatamente às arestas do grafo definido no Capítulo 2: soquete entre CPU e placa-mãe, padrão de memória entre placa-mãe e RAM, e capacidade energética da fonte em relação ao TDP da CPU e ao TDP da GPU, respectivamente.

7.4 Camada de Lógica de Negócio

A camada de lógica de negócio é o núcleo inteligente do sistema e concentra a contribuição técnica deste trabalho. Foi implementada em NestJS, *framework* escolhido por organizar a aplicação em módulos, *controllers* e *services*, estrutura que favorece a separação de responsabilidades e alinha-se diretamente aos princípios da arquitetura em camadas discutidos no Capítulo 2. Esta seção detalha, de dentro para fora, o isolamento do motor, a montagem do grafo de restrições, o algoritmo AC-3, o avaliador genérico de restrições e a geração das justificativas educativas.

7.4.1 Isolamento do Motor de Inferência

A decisão de projeto mais importante desta camada foi manter o motor de inferência inteiramente independente do *framework* e do mecanismo de persistência. Os arquivos que implementam o algoritmo AC-3 (`ac3.ts`) e a avaliação de restrições (`constraint-evaluator.ts`) não importam qualquer artefato do NestJS ou do Prisma, não contêm decoradores nem efeitos colaterais de entrada e saída, operando como funções puras sobre estruturas de dados recebidas por parâmetro.

Esse isolamento não é um detalhe de organização, mas o que garante três propriedades fundamentais do trabalho. Primeiro, o determinismo (RNF-10): sem qualquer dependência de estado externo ou de operações de entrada e saída, a mesma entrada produz invariavelmente a mesma saída. Segundo, a testabilidade: por não depender do *framework*, o motor pode ser exercitado por testes unitários que não exigem servidor nem banco de dados em execução. Terceiro, e mais importante para a tese central deste trabalho, a agnosticidade ao domínio: o motor não conhece o conceito de *hardware*, de soquete ou de TDP, opera exclusivamente sobre variáveis, domínios e restrições. Todo o conhecimento específico do domínio reside nos dados, e não no algoritmo, o que torna o motor, em princípio, aplicável a qualquer problema de configuração modelável como CSP.

7.4.2 Carregamento e Montagem do Grafo

A validação inicia-se no `CspService`, que, a cada requisição, carrega as restrições persistidas e as converte em uma representação interna independente do banco. Essa reconstrução a cada chamada é uma consequência direta da decisão de projeto pela ausência de estado no servidor: em conformidade com o princípio *stateless* (RNF-11), nenhuma informação de sessão é mantida entre requisições, e o grafo é montado a partir da base de conhecimento sempre que uma validação é solicitada. O Código 7.3 apresenta esse carregamento, no qual cada linha da tabela `Restricao` é mapeada para a estrutura interna `RestricaoInterna`, resolvendo, por meio das características associadas, quais categorias, as variáveis do CSP, a restrição conecta.

Código 7.3 – Carregamento das restrições a partir da base de conhecimento

```

1 private async carregarRestricoes(): Promise<RestricaoInterna[]> {
2   const linhas = await this.prisma.restricao.findMany({
3     include: {
4       caracteristica1: { select: { categoriaId: true } },
5       caracteristica2: { select: { categoriaId: true } },
6     },
7   });
8
9   return linhas.map((linha) => ({
10     id: linha.id,
11     variavelDemanda: linha.caracteristica1.categoriaId,
12     variavelCapacidade: linha.caracteristica2.categoriaId,
13     caracteristica1Id: linha.caracteristica1Id,
14     caracteristica2Id: linha.caracteristica2Id,
15     operador: linha.operador,
16     parametro: linha.parametro ?? undefined,
17     templateJustificativa: linha.templateJustificativa,
18   }));
19 }

```

Fonte: Elaborado pelo autor

Cada variável do CSP é representada por uma estrutura que associa a categoria ao seu domínio, mantido em um **Map** de componentes. A escolha do **Map** deve-se à necessidade de remover valores do domínio de forma eficiente durante a propagação, operação frequente no AC-3. Sobre o conjunto de restrições, o motor constrói uma **lista de adjacência**, conforme justificado teoricamente no Capítulo 2 como a representação mais adequada para grafos esparsos. A lista é materializada uma única vez no início da execução, indexando, para cada variável, os arcos direcionados que a alcançam, como mostra o Código 7.4.

Código 7.4 – Construção da lista de adjacência do grafo de restrições

```

1 function construirListaAdjacencia(
2   restricoes: RestricaoInterna[],
3 ): Map<string, Arco[]> {
4   const mapa = new Map<string, Arco[]>();
5   const adicionar = (chave: string, arco: Arco) => {
6     const lista = mapa.get(chave);
7     if (lista) lista.push(arco);
8     else mapa.set(chave, [arco]);
9   };
10 }

```

```
11   for (const r of restricoes) {
12       adicionar(r.variavelDemanda,
13           { origem: r.variavelCapacidade, destino: r.variavelDemanda,
14             restricao: r });
15       if (r.variavelDemanda !== r.variavelCapacidade) {
16           adicionar(r.variavelCapacidade,
17               { origem: r.variavelDemanda, destino: r.
18                 variavelCapacidade, restricao: r });
19       }
20   }
21   return mapa;
22 }
```

Fonte: Elaborado pelo autor

A opção pela lista de adjacência indexada, em vez de percorrer o conjunto completo de restrições a cada consulta, alinha a implementação à fundamentação teórica e assegura que a recuperação dos vizinhos de uma variável ocorra em tempo proporcional ao seu grau. Como cada restrição binária origina dois arcos direcionados, a lista preserva a orientação exigida pelo algoritmo, distinção necessária porque, no motor, a direção do arco determina qual característica é tratada como demanda e qual como capacidade.

7.4.3 O Algoritmo AC-3

O motor implementa o algoritmo AC-3 conforme descrito por Mackworth (1977) e Russell e Norvig (2010), decisão fundamentada no Capítulo 2: por operar por propagação de restrições, e não por enumeração exaustiva, o AC-3 evita a explosão combinatória que inviabilizaria a validação por força bruta (RNF-02). Uma fila é inicializada com todos os arcos direcionados; a cada iteração, um arco é retirado e submetido ao procedimento de revisão, que remove do domínio da variável de origem os valores sem suporte na variável de destino. Sempre que ocorre uma remoção, os arcos que incidem sobre a variável afetada são reinseridos na fila, propagando a mudança pela rede até o ponto de estabilidade. O Código 7.5 apresenta esse procedimento de revisão.

Código 7.5 – Procedimento de revisão de arco do algoritmo AC-3

```
1 function revisar(arco: Arco, variaveis: VariavelCSP[]):
    ValorRemovido[] {
2     const removidos: ValorRemovido[] = [];
3     const varOrigem = getVariavel(variaveis, arco.origem);
4     const varDestino = getVariavel(variaveis, arco.destino);
5     if (!varOrigem || !varDestino) return removidos;
6 }
```

```

7   const arcoReverso    = arco.origem === arco.restricao.
    variavelCapacidade;
8   const valoresDestino = Array.from(varDestino.dominio.values());
9   if (valoresDestino.length === 0) return removidos;
10
11  for (const v of Array.from(varOrigem.dominio.values())) {
12    const temSuporte = valoresDestino.some((w) =>
13      arcoReverso
14        ? avaliarRestricao(w, v, arco.restricao)
15        : avaliarRestricao(v, w, arco.restricao),
16    );
17    if (!temSuporte) {
18      varOrigem.dominio.delete(v.id);
19      removidos.push({
20        categoriaId:    varOrigem.categoriaId,
21        componenteId:   v.id,
22        componente:     v,
23        restricaoViolada: arco.restricao,
24        ancora:         valoresDestino[0],
25      });
26    }
27  }
28  return removidos;
29 }

```

Fonte: Elaborado pelo autor

Um aspecto de projeto merece destaque: cada valor removido não é simplesmente descartado, mas registrado em uma estrutura **ValorRemovido** que retém a restrição violada e um componente-âncora, o valor da variável vizinha que fundamenta o bloqueio. Essa decisão é o que torna possível, mais adiante, rastrear cada bloqueio até a restrição que o originou (RNF-13) e produzir a justificativa educativa correspondente, assegurando que nenhum bloqueio ocorra silenciosamente (RNF-14). Em outras palavras, a *explanation facility* não é um acréscimo posterior ao motor: a informação necessária para explicar é coletada no exato instante da poda.

7.4.4 Avaliação Genérica de Restrições

O suporte entre dois valores é decidido pela função **avaliarRestricao**, apresentada no Código 7.6, que aplica o operador definido na própria restrição, sem qualquer ramificação por tipo de característica ou de componente.

Código 7.6 – Avaliação genérica de uma restrição

```
1 export function avaliarRestricao(  
2   valorVar1: Componente,  
3   valorVar2: Componente,  
4   restricao: RestricaoInterna,  
5 ): boolean {  
6   const val1 = getValor(valorVar1, restricao.caracteristica1Id);  
7   const val2 = getValor(valorVar2, restricao.caracteristica2Id);  
8   if (val1 === undefined || val2 === undefined) return true;  
9  
10  if (restricao.operador === 'IGUAL') {  
11    return val1 === val2;  
12  }  
13  const parametro = Number(restricao.parametro ?? '1');  
14  return Number(val2) >= Number(val1) * parametro;  
15 }
```

Fonte: Elaborado pelo autor

A ausência deliberada de qualquer condicional específica por domínio é o que concretiza, no nível do algoritmo, os requisitos RNF-07 e RNF-08: o mesmo código avalia a compatibilidade de soquete, o padrão de memória ou a capacidade energética, distinguindo esses casos apenas pelo operador e pelas características cadastradas na base de conhecimento. O operador **IGUAL** implementa as restrições binárias absolutas de soquete e de padrão de memória; o operador **MAIOR_OU_IGUAL** implementa a restrição quantitativa de capacidade energética, na qual o parâmetro cadastrado, por exemplo, 1,25, representa a margem de segurança aplicada sobre a demanda. Cada verificação de potência é realizada entre um par de variáveis, decisão que preserva a natureza estritamente binária do grafo de restrições; as implicações dessa modelagem são discutidas na Seção 7.8.

7.4.5 Geração das Justificativas Educativas

A *explanation facility*, característica que distingue o sistema de um mero mecanismo de filtragem, é implementada no **ExplanationsService**. Para cada valor removido pelo AC-3, o serviço formula uma justificativa em linguagem natural a partir do **templateJustificativa** associado à restrição violada, substituindo marcadores por valores concretos dos componentes envolvidos. Os marcadores suportados são **{comp1.nome}**, **{val1}**, **{comp2.nome}**, **{val2}** e **{val1_com_margem}**, este último aplicável às restrições quantitativas, no qual o consumo é apresentado já acrescido da margem de segurança.

A decisão de manter o texto de cada explicação inteiramente no dado cadastrado, e não no código, é coerente com toda a arquitetura genérica adotada: assim como nenhuma regra de compatibilidade está embutida no motor, nenhum termo técnico do domínio

está embutido no serviço de explicações. Toda a dimensão educativa emerge da base de conhecimento. Como consequência, ao bloquear um módulo DDR4 diante de uma placa-mãe que suporta exclusivamente DDR5, o sistema não apenas remove a opção, mas comunica a regra física violada em linguagem acessível, transformando o bloqueio em oportunidade de aprendizagem (RF-10 a RF-13), que é precisamente o objetivo educacional do trabalho.

7.5 Camada de Apresentação

A camada de apresentação foi implementada em Next.js, utilizando o *App Router* e a biblioteca React (Meta Platforms, Inc., 2025) em sua versão 19. A interface adota o formato *wizard*, conduzindo o usuário sequencialmente pelas cinco categorias de componentes, decisão de projeto que reduz a carga cognitiva do usuário leigo ao decompor a montagem em etapas discretas. O estado da configuração, etapa atual, seleções realizadas e resultado da última validação, é mantido no próprio cliente por meio do mecanismo de estado do React, sem persistência no servidor, opção coerente com o caráter *stateless* da API e com o requisito RNF-05, que dispensa cadastro ou autenticação.

A comunicação com a camada de lógica de negócio é encapsulada em um módulo cliente (`lib/api.ts`) que expõe as operações de listagem de categorias, listagem de componentes e validação da configuração. Esse encapsulamento concentra em um único ponto o conhecimento sobre como falar com o servidor, isolando o restante da interface dos detalhes de comunicação. A cada alteração de seleção, a interface envia o estado completo à API e recebe, em resposta, os domínios atualizados e as justificativas dos bloqueios, atualizando a exibição em tempo real.

A decisão de interface mais alinhada ao objetivo educativo é a de não ocultar os componentes incompatíveis. Em vez de removê-los da listagem, comportamento típico dos mecanismos de filtragem, eles permanecem visíveis, com sinalização visual de bloqueio, e a justificativa técnica correspondente é revelada quando o usuário interage com o componente, atendendo aos requisitos RF-03 e RNF-14. Ao término da seleção de todas as categorias, a interface apresenta a tela de resumo, que consolida a configuração montada, exibe seu status geral de compatibilidade e o consumo energético estimado, e registra o tempo de execução da validação.

Para tornar concreto o encadeamento entre as camadas, convém percorrer o fluxo completo de uma interação. Quando o usuário seleciona um componente, a interface registra a escolha no estado local e envia o estado completo das seleções à API. No servidor, o `CspService` carrega as restrições e os domínios da base de conhecimento, monta o grafo e executa o AC-3, que propaga as restrições e poda os valores sem suporte; para cada valor podado, o `ExplanationsService` formula a justificativa correspondente. A

resposta, domínios atualizados, justificativas e tempo de execução, retorna à interface, que reatualiza a listagem, marcando os componentes recém-bloqueados e disponibilizando suas explicações. Todo esse ciclo ocorre a cada seleção, o que produz a validação incremental e em tempo real prevista nos requisitos.

7.6 A API REST

A comunicação entre a camada de apresentação e a de lógica de negócio ocorre exclusivamente por meio de uma API REST (Fielding, 2000), decisão que concretiza o princípio de separação cliente-servidor discutido no Capítulo 2 e assegura que interface e motor possam evoluir de forma independente. A API é exposta sob o prefixo `/api` e protegida por validação de entrada, expondo os três recursos apresentados no Quadro 36.

Quadro 36 – Recursos expostos pela API REST

Método	Rota	Descrição
GET	<code>/api/categorias</code>	Lista as categorias ordenadas
GET	<code>/api/components/:categoriaId</code>	Lista componentes de uma categoria
POST	<code>/api/configurations/validate</code>	Valida a configuração e retorna domínios e justificativas

Fonte: Elaborada pelo autor

O recurso de validação é o cerne da API. Recebe, no corpo da requisição, o estado completo das seleções do usuário, representado como um mapeamento entre categorias e componentes escolhidos, e retorna uma resposta que reúne o indicador de consistência, os domínios atualizados de cada variável, as justificativas dos bloqueios e o tempo de execução em milissegundos. A decisão de que cada requisição carregue todo o contexto necessário ao seu processamento é o que permite à API dispensar qualquer sessão persistente, satisfazendo o princípio *stateless* (Fielding, 2000) e o requisito RNF-11, e conferindo ao servidor simplicidade operacional e escalabilidade horizontal.

7.7 Execução do Ambiente

O ambiente completo é colocado em execução por uma sequência de comandos que provisiona o banco em contêiner, aplica as *migrations*, popula a base de conhecimento e inicia, de forma orquestrada, o servidor e o cliente. O Turborepo coordena a execução simultânea das duas aplicações a partir da raiz do repositório, como mostra o Código 7.7.

Código 7.7 – Comandos de inicialização do ambiente de desenvolvimento

```
1 pnpm install          # instala as dependencias do monorepo
2 pnpm db:up            # sobe o PostgreSQL em container (docker
                        compose)
```

```
3 pnpm db:migrate      # aplica as migrations (prisma migrate dev)
4 pnpm db:seed         # popula a base de conhecimento (seed.ts)
5 pnpm dev             # inicia API e cliente (turbo run dev)
```

Fonte: Elaborado pelo autor

Em execução, o cliente responde na porta 3000, a API na porta 3001 e o banco de dados na porta 5432. As configurações sensíveis ao ambiente, a URL de conexão com o banco, a porta e o prefixo da API, a origem autorizada para requisições e o endereço da API consumido pelo cliente, são fornecidas por variáveis de ambiente, e não figuram no código-fonte, prática que separa a configuração do código e evita o versionamento de dados sensíveis.

7.8 Limitações Conhecidas da Implementação

Em conformidade com o rigor esperado de um trabalho de engenharia, registram-se as limitações conhecidas da implementação em seu estado atual, todas decorrentes de decisões conscientes de projeto e coerentes com o escopo definido para a etapa de Validação de Projeto.

A primeira limitação decorre da própria generalidade do motor. Como a correteude das validações depende do cadastro correto das restrições na base de conhecimento, uma restrição quantitativa cadastrada com as características de demanda e de capacidade invertidas seria avaliada na ordem trocada, podendo produzir um falso-positivo. Trata-se de uma característica inerente aos sistemas especialistas dirigidos a dados, nos quais a qualidade da inferência é indissociável da qualidade da base de conhecimento (Todd, 1992); a mitigação estrutural desse risco, por meio de validação de esquema no momento do cadastro, é identificada como trabalho futuro.

A segunda limitação refere-se à modelagem da restrição de capacidade energética. Cada verificação de potência é realizada de forma binária, confrontando individualmente o TDP da CPU e o da GPU com a capacidade da fonte, o que preserva a natureza estritamente binária do grafo de restrições adotada em todo o trabalho. O consumo agregado da configuração é calculado e apresentado ao usuário na tela de resumo como informação, mas não é modelado como uma restrição do CSP, uma vez que uma restrição sobre a soma dos consumos seria de ordem superior à binária. Essa delimitação é intencional e mantém a coerência entre a fundamentação teórica e a implementação.

Por fim, a estratégia de testes automatizados encontra-se parcialmente desenvolvida. O motor de inferência dispõe de testes unitários que asseguram seu determinismo e a correteude da propagação, mas os testes de integração da API e os testes da camada de apresentação estão previstos como atividade posterior, conforme registrado entre os

requisitos adiados na Seção ?? do Capítulo 5.

8 TESTE

Descrever nesse capítulo quais e como foram os testes realizados. Os testes podem ser apresentados na forma de casos de teste. Um caso de teste consiste em conjunto de detalhes necessários para se realizar um teste de software.

A seguir encontra-se um modelo para especificação de um caso de teste (CEDRO, 2021):

Título O título do caso de teste deverá ser sucinto, simples e autoexplicativo com informações para que o Analista de Teste saiba a validação a qual o teste se propõe. Exemplos: • Validar upload de arquivo; • Validar cadastro de usuário com perfil administrador; • Validar envio de ordem de compra.
Objetivo O objetivo do caso de teste é descrever o que será executado, fornecendo uma visão geral do teste que será realizado. Exemplos: • Verificar se realiza o upload do arquivo com as extensões permitidas; • Verificar se o cadastro é efetivado após preencher as informações corretamente; • Verificar se a ordem de compra é enviada informando o ativo, quantidade e preço.
Pré-Condição São condições necessárias para que o caso de teste consiga ser executado. Evitar que não tenha alguma informação necessária (Exemplo: solicitar a edição de um usuário em específico e na pré-condição não informar que o usuário deve estar cadastrado). Exemplos: • Usuário cadastrado e autenticado no sistema; • Ordem de compra enviada e executada; • Usuário com perfil Administrador.

Passos

Os passos são necessários para descrever todas as ações que o analista deve seguir durante a execução para chegar ao resultado esperado. Devendo iniciar com um verbo infinitivo (acessar, preencher, clicar, verificar) ou imperativo (acesse, preencha, clique, verifique). Exemplos:

- Acessar a tela Negociação > Boleta;
- Clicar no botão “Entrar”;
- Verificar se a edição foi salva no banco de dados;
- Preencher os campos do cadastro.

Resultados Esperados

Descrever o comportamento esperado do sistema após executar os passos detalhados. Informar os verbos no presente (valida, apresenta, recupera, retorna). Evitar frases como “O sistema **deve** retornar a mensagem...”, prefira usar “O sistema retorna a mensagem...” para não deixar nenhuma dúvida do resultado esperado. Exemplos:

- Sistema apresenta a tela de edição com os campos preenchidos.;
- A ordem é enviada e executada com o preço informado;
- O cadastro é salvo no banco de dados.

Os casos de teste podem ser especificados usando uma ferramenta de software. Caso este seja o caso, esta seção deve apresentar qual a ferramenta e foi utilizada no desenvolvimento deste trabalho. A Figura 16 apresenta um exemplo de caso de teste especificado usando a ferramenta Testlink.

Figura 16 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink

Caso de Teste			
 CT-AUT-140: Validar cadastro de cliente			
Versão 1  			
Objetivo do Teste:			
Verificar se realiza o cadastro do cliente informando nome, CPF e telefone.			
Pré-condições			
1. Usuário cadastrado e autenticado no Portal ABC; 2. Usuário com perfil Administrador; 3. Possuir CPF válido.			
  #	Ações do Passo	Resultados Esperados:	 
1	1 - Acessar a tela de cadastro de cliente no Portal ABC: Menu principal > Cadastros > Cliente; 2 - Preencher os campos com dados válidos: • Nome • CPF • Telefone 3 - Clicar em Salvar; 4 - Verifique se o cadastro do cliente foi salvo no Banco de dados.	1 - Sistema exibe a tela de cadastro de cliente com os campos vazios; 2 - Após salvar o cadastro exibe a mensagem de sucesso: "Cliente cadastrado."; 3 - O registro do cliente é salvo no Banco de dados.	 

Fonte: Elaborada pelo autor

9 IMPLANTAÇÃO

Em qual servidor o sistema está implantado, em qual servidor está a aplicação e em qual servidor está o banco de dados, se foi feito registro de domínio, como deve ser feita a implantação e configuração do sistema, etc etc etc.

Caso o sistema esteja em execução em um ou mais clientes, apresentar nesse capítulo como foi a implantação. Sendo necessário, podem ser usados diagramas UML, como o diagrama de atividades, para apresentar as tarefas de implantação.

10 RESULTADOS E DISCUSSÃO

Texto dos resultados.

11 CONCLUSÕES PARCIAIS

Este trabalho propôs o desenvolvimento de um sistema especialista educacional para validação de compatibilidade de hardware, fundamentado no formalismo de Problemas de Satisfação de Restrições e no algoritmo de propagação AC-3, com a finalidade de não apenas detectar incompatibilidades entre componentes, mas de comunicar ao usuário leigo a razão técnica de cada uma em linguagem acessível. Nesta etapa de Validação de Projeto, apresentam-se as conclusões parciais obtidas a partir do que já foi elicitado, modelado e implementado.

O objetivo geral mostrou-se plenamente viável ao longo do desenvolvimento. A modelagem do problema de compatibilidade de hardware como um CSP revelou-se adequada, uma vez que as três categorias de restrições estudadas, soquete, padrão de memória e capacidade energética, ajustaram-se naturalmente à estrutura de variáveis, domínios e restrições. Sobre essa modelagem, o algoritmo AC-3 foi implementado como motor de inferência funcional, capaz de propagar restrições de forma incremental e podar os domínios das variáveis a cada seleção do usuário, sem recorrer à avaliação exaustiva por força bruta.

Quanto aos objetivos específicos, os resultados parciais são igualmente favoráveis. A representação de conhecimento adotada, baseada em um modelo genérico de metadados, permitiu dissociar o motor de inferência do domínio de aplicação, de modo que as regras de compatibilidade residem inteiramente na base de conhecimento e não no código. Essa decisão concretizou os requisitos de expansibilidade estabelecidos, pois a inclusão de novas categorias, características ou restrições depende apenas da inserção de registros, sem alteração do algoritmo. O mecanismo de explicação também foi implementado, gerando para cada incompatibilidade uma justificativa em linguagem natural a partir de modelos de texto associados às restrições, o que assegura que nenhum bloqueio ocorra de forma silenciosa.

A aplicação do motor ao domínio de compatibilidade de hardware, tomado como estudo de caso, confirmou a validade da abordagem. A base de conhecimento foi populada com componentes reais das plataformas AM4 e AM5, e os cenários de teste demonstraram que o sistema identifica corretamente as incompatibilidades de soquete, memória e potência, apresentando as justificativas correspondentes. A interface no formato *wizard*, por sua vez, materializou o caráter educativo da solução ao manter os componentes incompatíveis visíveis e acompanhados de suas explicações, em vez de simplesmente ocultá-los.

Do ponto de vista do problema de pesquisa, os resultados parciais indicam que é possível construir um sistema especialista que valide a compatibilidade de hardware por meio de um motor formal e que, ao mesmo tempo, ensine o usuário comunicando

a razão de cada incompatibilidade. A articulação entre a validação por propagação de restrições e a capacidade de explicação herdada dos sistemas especialistas mostrou-se não apenas possível, mas coerente, atendendo simultaneamente à dimensão técnica e à dimensão pedagógica que motivaram o trabalho.

Reconhecem-se, contudo, as limitações próprias do estágio atual. A corretude das validações depende do cadastro correto das restrições na base de conhecimento, característica inerente aos sistemas dirigidos a dados. A validação da capacidade energética é realizada de forma binária, componente a componente, preservando a natureza do grafo de restrições, ao passo que o consumo agregado da configuração é apresentado apenas como informação. Além disso, a estratégia de testes automatizados encontra-se parcialmente desenvolvida, com os testes de integração e de interface previstos como atividade posterior.

Para a etapa de Avaliação Final, planeja-se a ampliação da base de conhecimento para contemplar outras plataformas e categorias de componentes, a complementação da estratégia de testes automatizados e a realização de uma avaliação da eficácia educativa da solução junto a usuários. As conclusões parciais aqui apresentadas confirmam que os objetivos propostos são alcançáveis e que o sistema desenvolvido cumpre, até o presente estágio, os propósitos técnico e educacional que fundamentaram este trabalho.

12 CRONOGRAMA

Segue abaixo o cronograma das atividades que serão executadas até a Avaliação Final de TCC.

Obs: Para facilitar, crie o cronograma usando o modelo do Word contido no projeto (imagens/templateCronograma.docx), ou qualquer outro *software*, salve a imagem e atualize o arquivo imagens/cronograma.png.

		Meses											
		JAN	FEV	MAR	ABR	MAI	JUN	JUL	AGO	SET	OUT	NOV	DEZ
Atividades	1	X	X										
	2		X	X	X								
	3			X	X	X							
	4			X	X								
	5	X	X	X	X	X	X	X	X	X	X	X	X

1. Descrição da atividade 1;
2. Descrição da atividade 2;
3. Descrição da atividade 3;
4. Descrição da atividade 4;
5. Descrição da atividade 5.

Obs: Este capítulo deve ser elaborado e estar contido em trabalhos de graduação para a Validação de Projeto de TCC. Na Avaliação Final de TCC este capítulo não deve existir, visto que não haverá atividades após a Avaliação Final.

REFERÊNCIAS

ADVANCED MICRO DEVICES. **Meet the New AMD Socket AM5 Platform**. [S. l.: s. n.], 2022. AMD Partner Insights. Disponível em: <https://www.amd.com/en/partner/articles/socket-am5-change-game.html>. Acesso em: 4 abr. 2026. Citado nas pp. 17, 21, 24.

ANDERSON, John R. *et al.* Cognitive Tutors: Lessons Learned. **The Journal of the Learning Sciences**, v. 4, n. 2, p. 167–207, 1995. DOI: 10.1207/s15327809jls0402_2. Citado na p. 33.

BARREDO ARRIETA, Alejandro *et al.* Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI. **Information Fusion**, v. 58, p. 82–115, 2020. DOI: 10.1016/j.inffus.2019.12.012. Citado nas pp. 32, 38, 68.

CLANCEY, William J. The Epistemology of a Rule-Based Expert System: A Framework for Explanation. **Artificial Intelligence**, v. 20, n. 3, p. 215–251, 1983. DOI: 10.1016/0004-3702(83)90008-5. Citado nas pp. 16, 17, 31–34, 37–39, 44, 68.

CORMEN, Thomas H. *et al.* **Introduction to Algorithms**. 2. ed. Cambridge: The MIT Press, 2001. Citado nas pp. 24, 26, 27.

DBEAVER CORP. **DBeaver: Universal Database Tool**. [S. l.: s. n.], 2025. Ferramenta de administração de banco de dados. Disponível em: <https://dbeaver.io>. Acesso em: 4 abr. 2026. Citado na p. 89.

DECHTER, Rina. **Constraint Processing**. San Francisco: Morgan Kaufmann, 2003. Citado nas pp. 23, 29.

DIESTEL, Reinhard. **Graph Theory**. 5. ed. Berlin: Springer, 2017. Disponível em: <https://diestel-graph-theory.com>. Acesso em: 4 abr. 2026. Citado na p. 24.

DOCKER, INC. **Docker: Accelerated Container Application Development**. [S. l.: s. n.], 2025. Plataforma de containerização. Disponível em: <https://www.docker.com>. Acesso em: 4 abr. 2026. Citado na p. 89.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado) – University of California, Irvine. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 4 abr. 2026. Citado nas pp. 35, 36, 40, 67, 87, 99.

- FONTELLES, Mauro José *et al.* Metodologia da pesquisa científica: Diretrizes para a elaboração de um protocolo de pesquisa. **Revista Paraense de Medicina**, v. 23, n. 3, p. 1–8, 2009. Disponível em: <http://files.bvs.br/upload/S/0101-5907/2009/v23n3/a1967.pdf>. Citado na p. 39.
- FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002. Citado nas pp. 34–36, 40, 41, 87.
- GERHARDT, Tatiana Engel; SILVEIRA, Denise Tolfo. **Métodos de Pesquisa**. Porto Alegre: Editora da UFRGS, 2009. Citado na p. 40.
- INTEL CORPORATION. **ATX Specification Version 3.0**. Santa Clara, 2022. Disponível em: <https://edc.intel.com>. Acesso em: 4 abr. 2026. Citado na p. 22.
- INTEL CORPORATION. **LGA1851 and LGA1700 Socket Incompatibility**. [S. l.: s. n.], 2023. Intel Support Knowledge Base. Article ID: 000099723. Disponível em: <https://www.intel.com/content/www/us/en/support/articles/000099723/processors.html>. Acesso em: 4 abr. 2026. Citado nas pp. 17, 21.
- JEDEC SOLID STATE TECHNOLOGY ASSOCIATION. **JESD79-5B: DDR5 SDRAM Standard**. Arlington, 2020. Disponível em: <https://www.jedec.org>. Acesso em: 4 abr. 2026. Citado nas pp. 17, 21, 24.
- JUKNA, Stasys *et al.* On P versus $NP \cap co-NP$ for Decision Trees and Read-Once Branching Programs. **Computational Complexity**, v. 8, n. 4, p. 357–370, 1999. DOI: 10.1007/s000370050005. Citado na p. 27.
- KUMAR, Vipin. Algorithms for Constraint-Satisfaction Problems: A Survey. **AI Magazine**, v. 13, n. 1, p. 32–44, 1992. DOI: 10.1609/aimag.v13i1.976. Citado nas pp. 24, 28, 29.
- LARMAN, Craig. **Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**. 3. ed. Upper Saddle River: Prentice Hall, 2004. Citado na p. 73.
- MACKWORTH, Alan K. Consistency in Networks of Relations. **Artificial Intelligence**, v. 8, n. 1, p. 99–118, 1977. DOI: 10.1016/0004-3702(77)90007-8. Citado nas pp. 17, 29, 30, 39, 40, 43, 51, 95.
- META PLATFORMS, INC. **React: The Library for Web and Native User Interfaces**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://react.dev>. Acesso em: 4 abr. 2026. Citado na p. 98.
- MEUPC.NET. **Meupc.net: Monte seu PC**. [S. l.: s. n.], 2024. Plataforma web. Disponível em: <https://www.meupc.net>. Acesso em: 4 abr. 2026. Citado nas pp. 16, 17, 37.

MICROSOFT CORPORATION. **TypeScript: JavaScript With Syntax For Types**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://www.typescriptlang.org>. Acesso em: 4 abr. 2026. Citado na p. 89.

MICROSOFT CORPORATION. **Visual Studio Code**. [S. l.: s. n.], 2025. Editor de código-fonte. Disponível em: <https://code.visualstudio.com>. Acesso em: 4 abr. 2026. Citado na p. 89.

MITTAL, Sanjay; FRAYMAN, Felix. Towards a Generic Model of Configuration Tasks. *In: PROCEEDINGS of the 11th International Joint Conference on Artificial Intelligence (IJCAI-89)*. Detroit: Morgan Kaufmann, 1989. p. 1395–1401. Citado na p. 38.

NESTJS. **NestJS: A Progressive Node.js Framework**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://nestjs.com>. Acesso em: 4 abr. 2026. Citado na p. 90.

NWANA, Hyacinth S. An Introduction to Intelligent Tutoring Systems. **The Knowledge Engineering Review**, v. 5, n. 4, p. 251–276, 1990. DOI: 10.1017/S0269888900005014. Citado na p. 38.

OPENJS FOUNDATION. **Node.js**. [S. l.: s. n.], 2025. Ambiente de execução JavaScript. Disponível em: <https://nodejs.org>. Acesso em: 4 abr. 2026. Citado na p. 89.

PAPADIMITRIOU, Christos H.; YANNAKAKIS, Mihalís. On Limited Nondeterminism and the Complexity of the V-C Dimension. **Journal of Computer and System Sciences**, v. 53, n. 2, p. 161–170, 1996. DOI: 10.1006/jcss.1996.0058. Citado na p. 26.

PCPARTPICKER LLC. **PC Part Picker: System Builder**. [S. l.: s. n.], 2024. Plataforma web. Disponível em: <https://pcpartpicker.com>. Acesso em: 4 abr. 2026. Citado nas pp. 16, 17, 37.

PNPM. **pnpm: Fast, Disk Space Efficient Package Manager**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://pnpm.io>. Acesso em: 4 abr. 2026. Citado na p. 90.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: The World's Most Advanced Open Source Relational Database**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://www.postgresql.org>. Acesso em: 4 abr. 2026. Citado na p. 89.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH, 2016. Citado na p. 86.

PRISMA DATA, INC. **Prisma ORM: Next-generation Node.js and TypeScript ORM**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://www.prisma.io>. Acesso em: 4 abr. 2026. Citado na p. 91.

- RICHARDSON, Leonard; RUBY, Sam. **RESTful Web Services**. Sebastopol: O'Reilly Media, 2007. Citado na p. 35.
- RUSSELL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3. ed. Upper Saddle River: Prentice Hall, 2010. Citado nas pp. 17, 22, 24, 25, 27–31, 39, 43, 95.
- SABIN, Daniel; WEIGEL, Rainer. Product Configuration Frameworks: A Survey. **IEEE Intelligent Systems**, v. 13, n. 4, p. 42–49, 1998. DOI: 10.1109/5254.708432. Citado nas pp. 37, 38.
- SIPSER, Michael. **Introduction to the Theory of Computation**. 3. ed. Stamford: Cengage Learning, 2012. Citado nas pp. 17, 26, 27.
- SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado nas pp. 48, 49, 61, 69.
- TODD, Bryan S. **An Introduction to Expert Systems**. Oxford, 1992. Citado nas pp. 31, 32, 100.
- VERCEL INC. **Next.js: The React Framework for the Web**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://nextjs.org>. Acesso em: 4 abr. 2026. Citado na p. 90.
- VERCEL INC. **Turborepo: High-performance Build System for JavaScript and TypeScript**. [S. l.: s. n.], 2025. Documentação oficial. Disponível em: <https://turbo.build/repo>. Acesso em: 4 abr. 2026. Citado na p. 90.
- WANG, Cixiao; WU, Feng. An Expert System Approach to Support Blended Learning in Context-Aware Environment. In: CHEUNG, Simon K. S. *et al.* (ed.). **Blended Learning: Enhancing Learning Success. Proceedings of the 11th International Conference on Blended Learning (ICBL 2018)**. Cham: Springer, 2018. v. 10949. (Lecture Notes in Computer Science), p. 24–35. DOI: 10.1007/978-3-319-94505-7_3. Citado na p. 38.
- WOOLF, Beverly Park. **Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning**. San Francisco: Morgan Kaufmann, 2008. Citado na p. 33.